

Rowhammer on Raspberry Pi's
**An Analysis of Rowhammer Attack Viability on Raspberry Pi 4B and
Raspberry Pi 5**

B a c h e l o r t h e s i s

**University of Applied Science Hof
Faculty Computer Science
Degree Program Medieninformatik**

**Submitted to
Prof. Dr. Florian Adamsky
Alfons-Goppel-Platz 1
95028 Hof**

**Submitted by
Jakob Drescher
Mtr. Nr.: 00498922
Steinäcker 10
95191 Leupoldsgrün**

Hof, February 24, 2026

Abstract

Low-cost single-board computers like the Raspberry Pi series, especially Raspberry Pi 4 and 5, are compact, energy-efficient platforms designed to deliver versatile computing across a wide range of applications. They rely on main memory to provide fast, temporary storage for actively processed data. Each bit of data in Dynamic Random-Access Memory (DRAM) is stored in a single memory cell that represents a binary value through the presence or absence of electrical charge in capacitor [87, 32, 82]. As DRAM cell dimensions continue to shrink in order to increase memory density within limited physical space, technological progress enhances capacity and efficiency while simultaneously amplifying susceptibility to disturbance effects. DRAM must be able to operate reliably without compromising integrity due to charge loss. Repeated gate-induced drain- and subthreshold leakage [87, 27] can cause sufficient charge loss and disturbance, ultimately leading memory cells to flip from 0 to 1 or vice versa. Such Software-induced fault, namely as Rowhammer, deliberately exploit these effects by triggering accelerated charge loss and disturbance errors, thereby corrupting memory contents in a way that can enable privilege escalation and other severe security consequences. In this paper, we systematically evaluate the feasibility of mounting rowhammer on modern Raspberry Pi platforms. To enable uncached accesses to DRAM, we examined multiple cache bypass techniques, of which cache eviction strategies and ARM cache maintenance instructions proved effective. Furthermore, we performed targeted accesses to specific DRAM banks and rows by reverse engineering the DRAM address mapping using the DRAMA approach and subsequently verifying the derived mappings. To outrun DRAM refresh and circumvent Target Row Refresh (TRR)-based mitigation's, we ported and adapted TRRespass to identify efficient hammering access patterns capable of inducing bitflips to test the vulnerability. Under the evaluated conditions, we could not observe any susceptibility on modern Raspberry Pi models varying in model and ram configuration to Rowhammer. Factors such as potential TRR behavior and the influence of DRAM page policies preclude a definitive claim of immunity. The results show that DRAM in modern Raspberry Pi models remains intact under tested Rowhammer conditions, demonstrating no observable compromise of memory integrity.

Keywords: *Rowhammer, DRAM, Raspberry Pi, Address Mapping, TRR*

Contents

1	Introduction	1
2	Background	3
2.1	DRAM Architecture	3
2.2	Refresh Behavior	4
2.3	Rowhammer Bug	6
2.4	Hammering Techniques	6
2.5	CPU Caches	7
3	Methodology	9
3.1	Research Design	9
3.2	Evaluation	10
4	Overview of Challenges	11
5	Direct DRAM Activation	12
5.1	Non-temporal Accesses	12
5.2	Uncached Memory	13
5.3	Cache Eviction	13
5.3.1	Cache Attacks	14
5.3.2	Feasibility Constraints	19
5.4	Flush Cache	20
5.4.1	Instruction Types	21
5.4.2	Cache Behavior	22
6	Address Mapping Reconstruction	24
6.1	Methods for Inferring DRAM Organization	24
6.1.1	Knock-Knock Approach	25
6.1.2	Row Hit, Row Conflict Distinction	25
6.1.3	DRAMA Approach	28
6.2	Verification	29
7	Outpacing DRAM Refresh	32
7.1	Mitigations	32
7.1.1	Software-based Defenses	32
7.1.2	Hardware-based Defenses	33
7.1.3	Constraints	33
7.2	TRR Bypass	34
8	Related Work	36
9	Discussion and Limitations	38
10	Conclusion	40
A	Listings	43

List of Figures

1	DRAM hierarchical architecture	4
2	LPDDR physical structure [79]	4
3	Temperature-based Refresh Rates [48]	5
4	Hammering Techniques	7
5	Memory Hierarchy	8
6	<i>Flush+Reload</i> - 4-way (columns) cache with 8 sets (rows) [62]	14
7	Results of <i>Flush+Reload</i>	15
8	Results of <i>Evict+Time</i>	17
9	<i>Prime+Probe</i> - 4-way (columns) cache with 8 sets (rows) [62]	18
10	Results of <i>Prime+Probe</i>	18
11	Results of <i>Evict+Reload</i>	19
12	Point of Coherency and of Unification [11]	21
13	Types of access patterns in DRAM	26
14	Row hit and -conflict histograms	27
15	Frequency of detected combinations for Raspberry Pi's 4	28
16	Frequency of detected combinations for Raspberry Pi's 5	29
17	Timing Histogram of Raspberry Pi 4 mapping	31

List of Abbreviations

CPU	Central Processing Unit	7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 19, 22, 23, 30, 35
DDR	Double Data Rate	2
DMA	Direct Memory Access	13
DRAM	Dynamic Random-Access Memory	II, 1, 2, 3, 4, 5, 6, 7, 8, 11, 12, 13, 15, 16, 19, 20, 21, 24, 25, 26, 28, 29, 30, 32, 33, 34, 35, 38, 40, 41
ECC	Error-Correcting Code	33
GPU	Graphics Processing Unit	37, 40
LLC	Last Level Cache	14
LPDDR	Low Power Double Data Rate	2, 38, 39, 41
MAC	Maximum Activate Count	34
NT	Non-Temporal	12, 13
OS	Operating System	35
PoC	Point of Coherency	20
PoU	Point of Unification	21
RAM	Random-Access Memory	7, 8
SRAM	Static Random-Access Memory	7, 8
TLB	Translation Lookaside Buffer	21
TRR	Target Row Refresh	II, 7, 11, 25, 33, 34, 38, 39, 40, 41

Listings

List of Tables

1	Evaluated Raspberry Pi configurations	10
2	Variety of cache bypass measurements in user- and kernelspace	22
3	Mapping comparison with prior work	29

1 Introduction

In modern computing systems Memory serves as a fundamental part, directly influencing overall performance, energy efficiency, and system reliability [65]. Ensuring that stored information cannot be altered without authorization is a basic security requirement, as unauthorized modifications would compromise both the integrity of computations and the system as a whole [94].

In DRAM, each bit of data is stored in a single memory cell composed of a capacitor and a transistor [32]. Over decades of technological progress, improvements in transistor and capacitor scaling have enabled DRAM to serve as the predominant technology used for main memory across a wide range of devices, from cloud servers and laptops to mobile phones [7]. The demand for increasingly large memory capacities within a limited physical footprint has driven continuous reductions in DRAM cell size and increases in density. These changes allow memory cells to operate with lower voltages and smaller charges, resulting in higher storage capacity and reduced energy consumption [24].

The extensive use of DRAM as main memory implies that potential security flaws within the technology can have broad impact on dependent computing systems [7]. Operations performed on a given memory address must remain strictly confined to that address and must not inadvertently affect the contents of other memory locations [65]. As memory technologies continue to scale toward smaller physical dimensions, DRAM chips become increasingly susceptible to disturbance effects, in which the behavior of a memory cell can unintentionally influence adjacent cells [65], causing the stored value of a DRAM cell to change without explicit modification, a phenomenon known as a bitflip [24].

In 2014, Kim et al. [87] demonstrated that DRAM is vulnerable to disturbance errors, showing that repeated activations of a memory row can induce charge leakage in neighboring cells. Such leakage may alter stored values, which in turn can be exploited to escape sandboxes or escalate privileges on a local system [32]

Initial verification of the Rowhammer phenomenon was conducted using FPGA-based memory controllers. Later studies extended these attacks to native x86 systems as well as ARM-based platforms [32]. Early speculation suggested that Rowhammer attacks on ARM devices might be infeasible, largely due to the presumed slower response of ARM memory controllers. Nonetheless, this assumption was later disproven when bitflips were successfully observed on ARM hardware [82].

Previous studies primarily focused on mobile phones or Chromebooks [6, 82, 67, 66]. Subsequently on older Raspberry Pi models, leaving it uncertain whether the latest generations exhibit similar vulnerabilities. This uncertainty is further compounded by the differences in

Low Power Double Data Rate (LPDDR) memory, which due to power and die area constraints, behaves differently from standard Double Data Rate (DDR) memory [66]. In the context of this Bachelor thesis, we want to analyze to what extent modern Raspberry Pi platforms, specifically the Raspberry Pi 4B and Raspberry Pi 5, are susceptible to bitflips when subjected to Rowhammer access patterns.

Outline. The work is divided into nine further sections. Section 2 "*Background*" introduces cache and DRAM architectures and general information of the Rowhammer bug and techniques. Section 3 "*Methodology*" describes the experimental design, measurement procedures, and evaluation criteria. Section 4 "*Overview of Challenges*" describes the fundamental techniques required to execute Rowhammer attacks effectively, including uncached, targeted, and fast access patterns by analyzing and evaluating the corresponding experiments of these primitives. Section 5 "*Direct DRAM Activation*", section 6 "*Address Mapping Reconstruction*", and section 7 "*Outpacing DRAM Refresh*" build on this overview and examine these techniques in greater detail. Section 8 "*Related Work*" reviews prior research on Rowhammer attacks. Section 9 "*Discussion and Limitations*" examines the implications of the results and the study's constraints. Section 10 "*Conclusion*" summarizes the findings and outlines potential directions for future work.

2 Background

2.1 DRAM Architecture

The core problem of Rowhammer is the underlying vulnerability in DRAM, which is widely employed as the primary memory technology in computers, smartphones, and servers [59, 27]. Its internal organization is based on a hierarchical structure of two-dimensional cell matrices arranged in rows and columns [27, 54, 82], with each cell storing a single bit of data using a capacitor that is controlled by an access transistor, where exactly one capacitor and one transistor are required per stored bit [27, 54, 59].

The widespread adoption of DRAM stems from its advantages, including low latency, high storage density, and cost-effectiveness per bit [27]. Despite these benefits, DRAM exhibits several inherent weaknesses. Over the years, DRAM capacity has increased significantly while the physical size of memory chips has remained largely unchanged leading manufacturers to integrate an increasing number of memory cells within the same die area to meet scaling memory demands [7, 94]. In addition, DRAM suffers from inherent data retention challenges caused by leakage mechanisms such as gate-induced drain leakage and subthreshold leakage [87, 27]. Disturbance errors can arise when sufficient interactions occur between circuit components, e.g., capacitors, transistors, or wires, that are intended to remain isolated [27]. The structural organization of DRAM, as illustrated in fig. 1 and fig. 2, is composed of several fundamental components that together enable data storage and access. At the lowest level, the transistor functions as a switch that controls access to the data stored within a memory cell [79, 54, 59]. The capacitor itself is responsible for storing a single bit of information in the form of an electric charge, where a charged state represents a logical one and an uncharged state represents a logical zero [37, 59]. A DRAM cell is therefore defined by the combination of exactly one transistor and one capacitor connected to both a wordline and a bitline, where the stored charge indicates a binary value [82].

The wordline runs horizontally across a row of cells and controls the access transistors, thereby enabling or disabling access to all cells within the corresponding row simultaneously [27]. In contrast, the bitline runs vertically along the same column and connects the cells to the sense amplifier, which is responsible detecting the small voltage differences on the bitline to a stable logic level for reliable data interpretation [35].

At a higher level of abstraction, memory cells are organized into banks, each forming a grid of rows and columns [79]. A DRAM bank consists of many memory cells, associated sense amplifiers, and a dedicated row buffer that stores the most recently accessed row for that specific bank [54, 87]. The presence of multiple banks enables memory requests to be

processed in parallel, thereby improving overall performance [59]. Beyond banks, DRAM modules are connected to the processor through channels, which are independent data buses that operate simultaneously to increase the memory bandwidth [54, 59]. Finally, a memory package encapsulates one or more DRAM dies and typically includes multiple channels, providing the physical and electrical interface between the memory subsystem and the rest of the system [79].

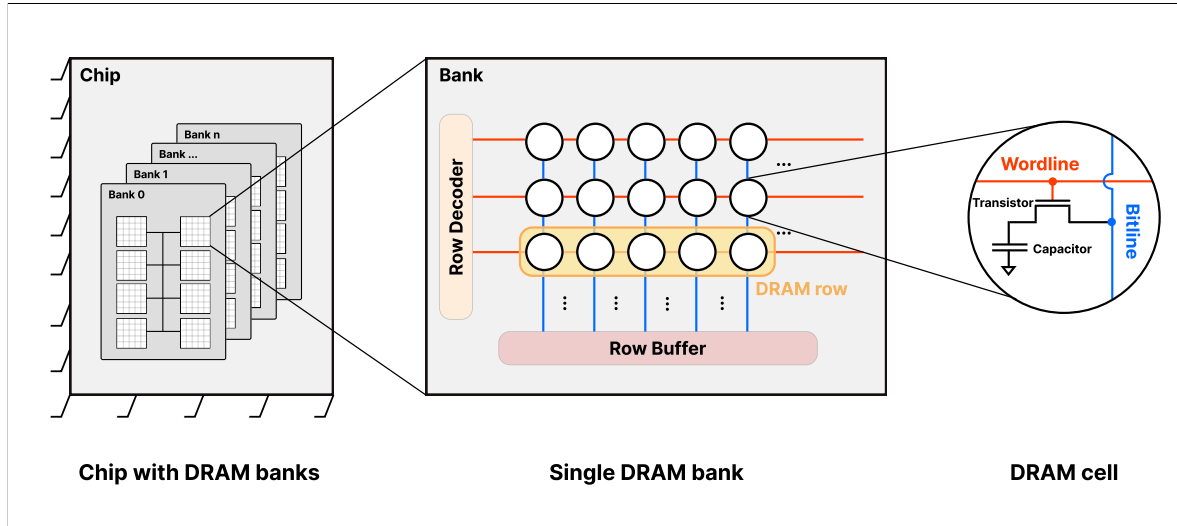


Figure 1: DRAM hierarchical architecture

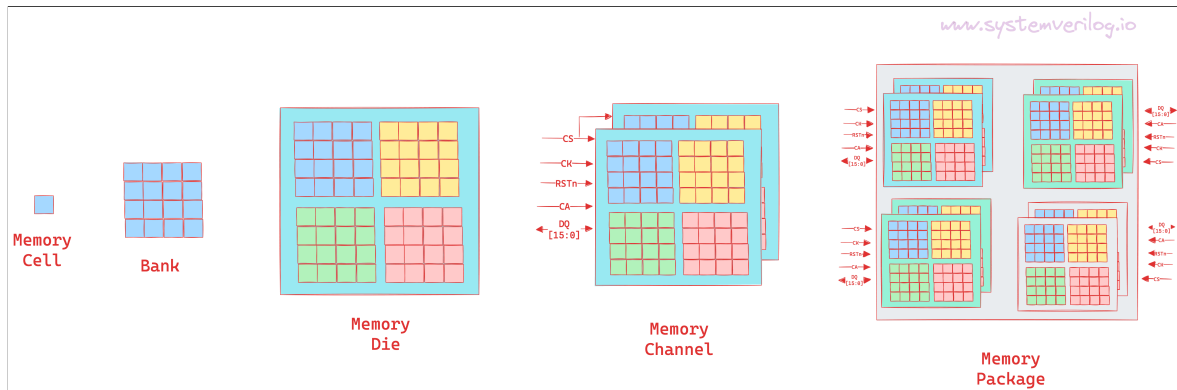


Figure 2: LPDDR physical structure [79]

2.2 Refresh Behavior

DRAM is a volatile memory, implying that all stored data is gone when the power supply is interrupted. Even while power is applied, stored information must be actively maintained, as the capacitors in DRAM cells gradually lose their charge over time [87]. To preserve data integrity, the memory controller therefore issues periodic refresh operations or regular read

and write commands, even when the memory is otherwise idle [54, 83, 59]. If a DRAM cell loses too much charge before being refreshed, a disturbance error may arise, resulting in incorrect data being stored [87].

During a refresh operation or a normal memory access, an entire row of DRAM cells is activated and loaded into the dedicated row buffer. This process is initiated by activating the corresponding wordline, which enables all access transistors in the selected row and connects the capacitors to their respective bitlines. The row buffer temporarily stores the data by sensing the charge from the cells, while the contents of the original row is overwritten. Subsequently, the data stored in the row buffer is restored back into the DRAM array, ensuring that the original information is preserved [27, 59, 54, 59].

When a memory access targets data that resides in a row already opened into the row buffer, the request can be served immediately without requiring additional row activation or precharge operations. This scenario is referred to as a row hit [59]. In contrast, a row conflict occurs when an access targets a different row within the same bank. In this case, the currently open row must first be closed by writing the contents of the row buffer back to the corresponding cells. Only after this operation can the new row be activated and fetched into the row buffer [59].

The refresh behavior of DRAM is governed by technology-specific timing parameters. For DDR4 memory, the default refresh window, denoted as t_{REFI} , is 64 ms. In comparison, LPDDR4 employs a shorter refresh interval of 32 ms and supports dynamic, temperature-based refresh adjustments that are configured through the MR4 Mode Register [48, 66]. The higher the temperature, the shorter the refresh interval. Conversely, the lower the temperature, the longer the refresh interval (see fig. 3 for details) [66, 48, 49].

MR4 OP[2:0]	Refresh rate	Max. No. of Pulled-in or Postponed REFab	Max. Interval between Two REFab	Max. No. of REFab within max(2xtREFI x Refresh Rate Multiplier, 16xtRFC)	Per bank Refresh
000B	Low Temp. Limit	N/A	N/A	N/A	N/A
001B	4x tREFI	8	9 x 4 x tREFI	16	1/8 of REFab
010B	2x tREFI	8	9 x 2 x tREFI	16	1/8 of REFab
011B	1x tREFI	8	9 x tREFI	16	1/8 of REFab
100B	0.5x tREFI	8	9 x 0.5 x tREFI	16	1/8 of REFab
101B	0.25x tREFI	8	9 x 0.25 x tREFI	16	1/8 of REFab
110B	0.25x tREFI	8	9 x 0.25 x tREFI	16	1/8 of REFab
111B	High Temp. Limit	N/A	N/A	N/A	N/A

Figure 3: Temperature-based Refresh Rates [48]

2.3 Rowhammer Bug

The DRAM architecture inherently introduces a security vulnerability, known as Rowhammer, which is independent of the operating system or virtualization layers [59, 82]. This software-induced hardware fault arises because increasing memory density places cells closer together, heightening the chance of interference between neighboring cells [87, 54, 94, 59]. Repeatedly accessing a specific row, termed the aggressor row, can induce sufficient disturbance in adjacent victim rows to flip bits before the internal refresh can restore them.

This vulnerability has enabled a wide range of powerful attacks, including privilege escalation [24, 21, 53, 82], sandbox escapes [21, 75, 53], ASLR defeating [69], cryptography breaking [50], and denial-of-service attacks [76, 64]. These attacks affect desktop systems [21, 53, 70], mobile devices [82, 6, 64, 70], and cloud environments [88, 50], even without exploiting traditional software vulnerabilities. Moreover, Rowhammer attacks can be triggered from diverse execution contexts, such as JavaScript environments [21], network-based attacks [64], or across virtual machines [50, 88].

As a result of inter-cell coupling effects, repeated activations of a row induce accelerated charge leakage in adjacent rows [59]. Building on this observation, prior work has proposed three possible underlying mechanisms to explain the phenomenon [87, 27, 65]. First, fluctuations in the voltage of a wordline may introduce interference into a neighboring wordline due to *electromagnetic coupling* effects [52, 18, 80]. This disturbance can momentarily activate the access transistors of the adjacent row, thereby promoting charge leakage [87]. Second, *bridging faults* in DRAM occur when conductive channels form between unrelated wires and/or capacitors [91, 92], and frequent toggling of a wordline can accelerate charge transfer between these bridged cells [87, 27]. Third, *hot-carrier injection* occurs when prolonged wordline toggling damages cells [68], potentially injecting hot-carriers into neighboring rows and increasing their charge leakage [87, 27].

2.4 Hammering Techniques

Based on these underlying physical mechanisms, several characteristic Rowhammer techniques have emerged. These techniques primarily differ in the number of neighboring rows they target and in their ability to bypass modern mitigation mechanisms.

A *single-sided* attack performs frequent accesses, or hammering, to only one row adjacent to a victim row. This approach is often combined with accesses to other rows within the same bank to induce bitflips in the neighboring victim row [39, 24, 82, 27], as can be seen from the fig. 4a.

In contrast, a *double-sided* attack repeatedly hammers two rows located on opposite sides of

a victim row, effectively sandwiching it (see fig. 4b) [24]. This increases the likelihood of inducing bitflips but requires partial knowledge of virtual-to-physical address mappings to correctly position the aggressor rows [27, 24, 82].

In fig. 4c, the *many-sided* attack extends this concept by employing multiple aggressor rows and distributed access patterns to evade TRR mitigation mechanisms that monitor only limited access behaviors [70, 6].

Finally, the *one-location* attack is a technique in which the attacker repeatedly accesses a single DRAM row at high frequency, as seen in fig. 4d. Unlike traditional attacks that hammer multiple rows, this approach relies on modern memory controllers that preemptively close rows instead of strictly following an open-page policy. By continuously re-opening the same row, the attacker induces repeated row activations and conflicts, which can lead to bitflips in rows adjacent to the hammered location, despite only targeting a single row [24, 64].

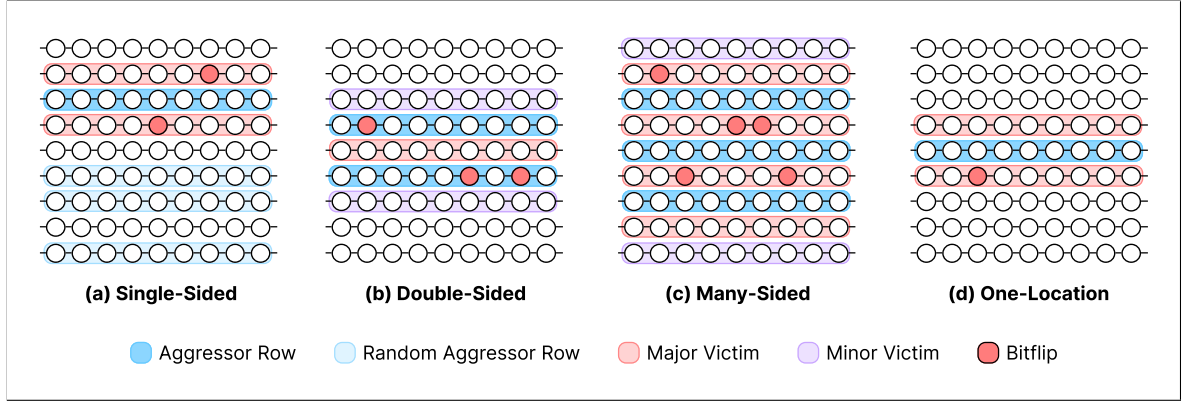


Figure 4: Hammering Techniques

2.5 CPU Caches

DRAM requires regular refreshing because it stores each bit as an electrical charge, which gradually leaks over time. This leakage necessitates refresh cycles, resulting in slower access times. However, DRAM provides high storage density at a relatively low cost [78]. In contrast, Static Random-Access Memory (Static Random-Access Memory (SRAM)) maintains each bit in a stable flip-flop circuit, eliminating the need for refresh operations. This characteristic makes SRAM faster and allows it to be physically located closer to the Central Processing Unit (CPU), though it is more expensive and offers lower storage capacity compared to DRAM [78].

CPU caches are constructed from SRAM to maximize access speed [71, 1]. A cache is a fast, specialized memory area designed to accelerate data retrieval [78]. It temporarily stores copies of frequently used data and instructions from main memory (Random-Access

Memory (RAM)) that the CPU is likely to access imminently [78, 71]. By holding data that the CPU may repeatedly access, caches serve as a buffer between the CPU and main memory, significantly reducing the time required for repeated data accesses [41, 71]. Cached data typically mirrors the contents of slower storage devices such as RAM or disks [78]. Common types of caches include CPU caches, memory caches, and browser caches [78]. Despite this, only CPU caches are relevant in the context of the Rowhammer attack.

Cache performance is characterized by hits and misses. A cache hit occurs when the CPU requests data already present in one of its cache levels. In this case, the cache provides the data immediately, bypassing the need to access main memory. This results in extremely low latency, as cache access is several orders of magnitude faster than DRAM access. Conversely, a cache miss happens when the requested data is not stored in the cache [1]. The CPU must then retrieve the data from DRAM, which introduces significantly higher latency due to slower access times. Once retrieved, the data is typically placed in the cache to accelerate potential future accesses. Cache hits improve overall system performance by exploiting temporal and spatial locality, ensuring that frequently or recently used data remains close to the CPU.

The memory hierarchy organizes storage components according to their speed, latency, and cost, as illustrated in fig. 5. Higher levels of the hierarchy, such as CPU caches, are smaller but faster, while lower levels are larger but significantly slower. Due to their proximity to the CPU and implementation using SRAM, caches provide access speeds approximately 10-100 times faster than DRAM [77].

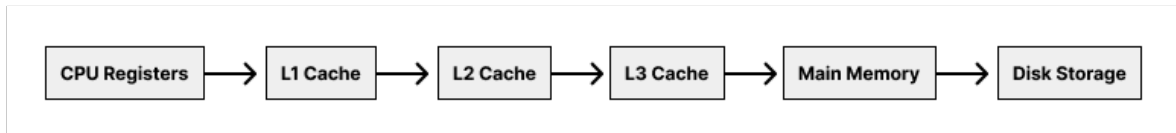


Figure 5: Memory Hierarchy

3 Methodology

This chapter describes the methodological framework applied in this thesis with the focus placed on theory, analysis and evaluation of the experiments. It outlines the research design, the applied methodological approach, as well as the procedures for data collection, analysis, and evaluation used to assess the susceptibility of modern Raspberry Pi models to Rowhammer attacks.

3.1 Research Design

This study is quantitative using experimental testing to confirm or refute specific hypotheses in order to obtain empirical evidence and clarity regarding the susceptibility of various modern Raspberry Pi models to Rowhammer attacks.

From a methodological perspective, the study primarily adopts a deductive approach. We tested existing Rowhammer theories on Raspberry Pi platforms by starting from established theoretical foundations, formulating hypotheses derived from these theories, and subsequently validating or refuting them through controlled experiments. In general, this deductive approach is characterized by the investigation of an existing theory, followed by the derivation of hypotheses, the execution of experiments, and the confirmation or falsification of the proposed assumptions.

In addition to the deductive core, the methodology incorporates an inductive approach in a supporting role. Experimental observations obtained during testing may reveal unexpected system behavior. Based on these measured results, new hypotheses or insights can be derived. In general terms, the inductive approach involves the derivation of theory from empirical research by identifying patterns in observations and formulating new hypotheses or theoretical considerations. Overall, we classify the study predominantly deductive and complement it with inductive elements that emerge from experimental findings.

The experimental setup included Raspberry Pi 4 models with 4 GB and 8 GB of RAM, as well as Raspberry Pi 5 models with 4 GB, 8 GB, and 16 GB of RAM (see table 1).

The Raspberry Pi 4 devices are equipped with a Quad Core ARMv8 Cortex-A72 CPU at 1.8 GHz, featuring 32 KB data and 48 KB instruction L1 cache per core and a 1 MB L2 cache, which collectively provide approximately 50 % higher performance than the Raspberry Pi 3B [74].

The Raspberry Pi 5 devices include a Quad Core ARMv8 Cortex-A76 CPU at 2.4 GHz, with 64 KB data and instruction L1 cache per core, 512 KB L2 cache per core, and a shared 2 MB L3 cache. In aggregate, the new features present in BCM2712 deliver a performance uplift

of 2-3x over Raspberry Pi 4 for common CPU or I/O-intensive use cases [74]. The selected ARM-based single-board computers were provided for the purpose of this study. The devices are comparable in terms of architecture, and overall system design, which enables a controlled comparison of Rowhammer susceptibility.

Table 1: Evaluated Raspberry Pi configurations

Platform	Processor	Architecture	SoC	DRAM
Raspberry Pi 4B	ARMv8	aarch64	Broadcom BCM2711	4 GB LPDDR4
Raspberry Pi 4B	ARMv8	aarch64	Broadcom BCM2711	8 GB LPDDR4
Raspberry Pi 5	ARMv8	aarch64	Broadcom BCM2712	4 GB LPDDR4x
Raspberry Pi 5	ARMv8	aarch64	Broadcom BCM2712	8 GB LPDDR4x
Raspberry Pi 5	ARMv8	aarch64	Broadcom BCM2712	16 GB LPDDR4x

The primary data sources consisted of related academic papers, providing the theoretical foundation. Supplementary technical documentation, datasheets, and glossaries to clarify hardware specifications and terminology. Additionally, GitHub repositories containing Rowhammer proof-of-concept implementations were utilized, adapted, and applied to the target devices.

Data collection involved the experimental execution of Rowhammer attacks on each Raspberry Pi model, combined with the observation of system behavior. Outcomes were systematically recorded and logged to facilitate further hypothesis formation. Data collection was conducted exclusively on the experimental devices to derive conclusions from the observed results and to support the generation of subsequent hypotheses.

3.2 Evaluation

To ensure the reliability and validity of results, experiments were repeated under controlled environmental conditions to minimize external interference. Observed outcomes were analyzed using statistical tests to assess the significance and consistency of the findings.

The experiments were conducted on the provided devices in a controlled setup. The study focuses on the responsible, academic investigation of Rowhammer vulnerabilities, with results intended solely for theoretical analysis and evaluation, avoiding any risk to other systems.

4 Overview of Challenges

The feasibility of Rowhammer attacks depends on a set of fundamental attack primitives, which define the conditions under which the attack can succeed [19, 36]. Specifically, three properties of memory accesses must simultaneously hold.

- C1. Direct DRAM Activation** First, memory accesses must be uncached in order to reach the DRAM directly, thereby bypassing the CPU cache. DRAM cells are only electrically stressed when the memory controller opens and closes an actual DRAM row. Consequently, this primitive ensures that each memory access physically toggles DRAM cells instead of being served from the CPU cache. This property is critical because Rowhammer relies on high-frequency, direct DRAM activations, which only occur when cache effects are avoided. If memory accesses remain cached, the effective hammering speed decreases by orders of magnitude and falls far below the threshold required to discharge charge in adjacent cells. Therefore, forcing cache misses, or alternatively using uncached memory regions, is mandatory to generate the activation frequency necessary for a successful attack.
- C2. Address Mapping Reconstruction** Second, memory accesses must be targeted such that they reach a specific intended physical DRAM row. This requires an understanding of DRAM mapping, as the attacker must determine how virtual addresses are translated to physical DRAM structures in order to select aggressor rows that surround a victim row. Achieving this goal necessitates reconstructing the DRAM address scheme. Only with accurate knowledge of this mapping can hammering be focused so that specific adjacent rows are repeatedly activated. Without correct mapping, hammering becomes largely probabilistic and rarely, if ever, results in disturbances to victim data.
- C3. Outpacing DRAM refresh** Third, memory accesses must be sufficiently fast to race against the next DRAM row refresh [36, 23]. The attack depends on hammering memory rows at a higher rate than DRAM refresh cycles can restore charge in adjacent cells. This, in turn, requires achieving high access throughput to the same DRAM rows with minimal delay. To sustain such high-frequency activation, optimal access parameters must be used, and the attack must often bypass mitigation mechanisms. In particular, defenses that throttle activation patterns or increase refresh activity, such as TRR or increased refresh rates, must frequently be circumvented to maintain sufficient hammering intensity.

5 Direct DRAM Activation

In the following, each attack primitive is discussed in turn, beginning with "*C1. Direct DRAM Activation*", which establishes the foundation for inducing repeated DRAM row activations. This challenge is realized by preventing data from being stored in or retrieved from the cache, thereby ensuring that all memory accesses are served directly from DRAM. To this end, the available techniques for bypassing the cache are first presented, followed by a focused discussion of two mechanisms that are particularly relevant for Raspberry Pi platforms: cache eviction and cache flushing.

We consider several cache-bypass techniques to enable the memory access behavior required for triggering Rowhammer [19, 36]. These techniques include the use of non-temporal memory accesses, the allocation of uncached memory regions, cache eviction strategies, and explicit cache flush operations. Prior work has shown that triggering Rowhammer on ARM-based devices presents substantial challenges, as successful exploitation requires rapid, repeated access to identical DRAM locations [94]. CPU caches significantly obstruct this process by intercepting repeated accesses, thereby preventing them from reaching DRAM. Consequently, mitigating cache effects becomes a central prerequisite. While x86 architectures provide well-established mechanisms - such as cache flushing via the `clflush` instruction, non-temporal accesses, and eviction-based techniques - [94] demonstrates that these approaches are either impractical or not well suited for direct adaptation to ARM platforms, thereby motivating the exploration of alternative, architecture-specific solutions. Since successful Rowhammer attacks require high-frequency access to DRAM, cache memory can significantly disrupt the attack by absorbing repeated accesses and reducing the number of DRAM activation's. Consequently, uncached access can be achieved through two complementary strategies, either by explicitly removing data from the cache or by preventing data from being cached in the first place [47].

5.1 Non-temporal Accesses

Prior work introduces a Rowhammer triggering technique on x86 platforms that leverages Non-Temporal (NT) memory instructions [75]. NT accesses are intended for data with low temporal locality, i.e., data that is unlikely to be reused in the near future and therefore should not occupy cache space, such as streaming data. By design, these accesses can reduce or avoid cache involvement, which has been shown to significantly facilitate Rowhammer attacks on x86 systems [75]. In contrast, on ARMv8 architecture NT load and store instructions (e.g., LDNP and STNP) merely act as advisory hints to the memory system rather than enforcing

uncached behavior [82]. Empirical results indicate that accessed data may still be cached in practice [94]. Moreover, multiple consecutive writes to the same address are collected within a single write-combining buffer, with only the last write being committed to DRAM [36]. This aggregation substantially reduces the effective access frequency to DRAM, rendering NT-based approaches unsuitable for achieving the activation rates required for Rowhammer attacks on Raspberry Pi devices.

5.2 Uncached Memory

Another approach involves allocating memory regions that are explicitly marked as uncached, for example through the use of ION, such that every access is served directly from DRAM without interacting with the cache [82]. This method is particularly suitable for ARM-based mobile architectures, such as ARMv7 and ARMv8, where cache flush instructions may require elevated privileges and non-temporal stores may still be subject to caching in practice [36]. A key advantage of this technique is that it operates entirely in unprivileged user space and does not require special permissions. Prior work describes how user-accessible Direct Memory Access (DMA) buffers exposed by the Android ION memory management interface can be leveraged to bypass CPU caches, thereby enabling Rowhammer triggering from user space [82, 94]. Even so, subsequent defenses, such as [81], have rendered Rowhammer bugs triggered in this way non-exploitable. Importantly, this technique is not applicable to Raspberry Pi systems, as the Android-specific ION framework is not available in standard Linux-based Raspberry Pi environments, which prevents the allocation or control of such uncached DMA buffers.

5.3 Cache Eviction

Cache eviction, in general, refers to techniques that force cache lines out of the cache by deliberately accessing memory addresses that belong to the same cache eviction set.

Cache attacks are a class of attacks that explicitly exploit the properties and effects of the cache hierarchy in order to gain information about the behavior or data of other programs or processes. Such attacks represent a powerful method for exploiting the memory hierarchy of modern system architectures [62]. To evict a specific address from the cache back to main memory, attackers can access congruent addresses that map to the same cache set, thereby forcing the cache replacement policy to evict the target address. The strategy that defines how these congruent addresses are accessed is referred to as an eviction strategy [62]. Over time, various methods have been proposed to reliably evict data from the cache.

5.3.1 Cache Attacks

The *Flush+Reload* attack was introduced by Gullasch et al. [26] and later extended by Yarom and Falkner [89]. It exploits shared memory between an attacker and a victim and provides very fine-grained resolution [20, 21]. The basic idea proposed in [26] is to use the `clflush` instruction to evict monitored memory locations from the cache and then check whether these locations have been reloaded after the victim program has executed a small number of instructions. Yarom and Falkner [89] further observed that the `clflush` instruction evicts cache lines from all cache levels, including the shared Last Level Cache (LLC), which enables *Flush+Reload* attacks even when the attacker and victim execute on different CPU cores [62]. *Flush+Reload* has been automated to exploit cache-based side channels and extended beyond cryptographic applications. In particular, Gruss et al. demonstrated a cache-based keylogger that infers keystrokes using *Flush+Reload* [25]. Additionally, Yarom et al. successfully recovered OpenSSL ECDSA nonces and, consequently, the private key using this technique [90]. The functionality of the *Flush+Reload* attack proceeds in several steps. First, the attacker maps a target binary, such as a shared library or executable, into their own virtual address space using a system call like `mmap`, as seen in fig. 6a. Next, a specific cache line associated with this shared memory region is evicted from the entire cache hierarchy, including the LLC, using a cache flush instruction (see fig. 6b). After the victim process is scheduled and executes (see fig. 6c), the attacker re-accesses the previously flushed memory location. The access latency is then measured to distinguish between a cache hit and a cache miss, as seen in fig. 6d. A low access time indicates that the data was reloaded into the cache, whereas a high access time indicates that the data was fetched from main memory. As seen in fig. 7, the

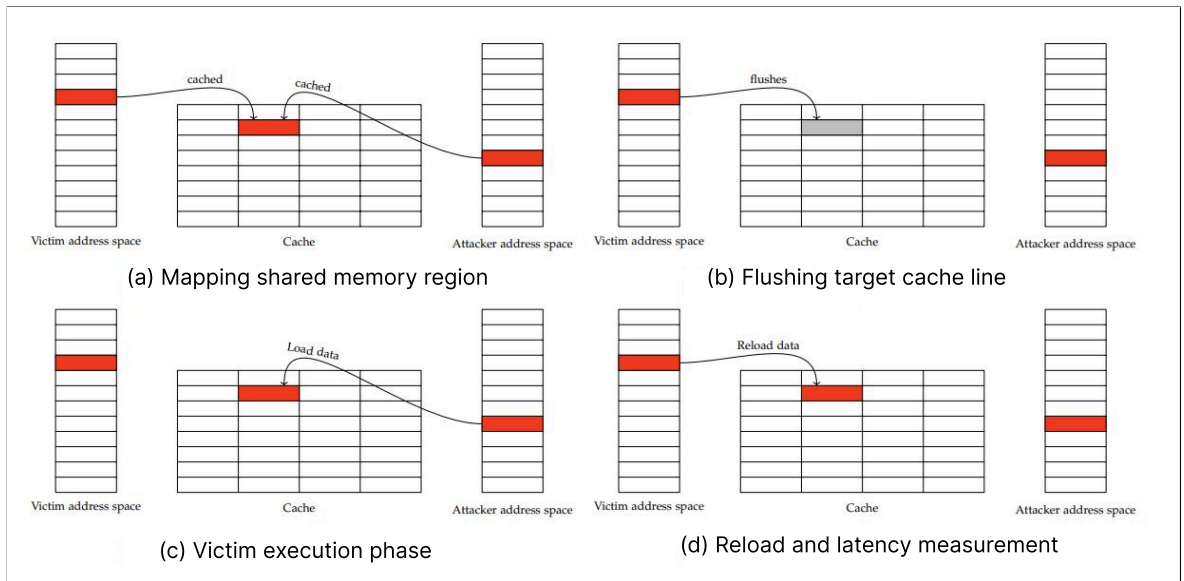


Figure 6: *Flush+Reload* - 4-way (columns) cache with 8 sets (rows) [62]

experimental results from *Flush+Reload* measurements on the Raspberry Pi 5 reveal three distinct peaks in access-time histograms: a cache-hit peak, an undefined intermediate peak, and a cache-miss or DRAM-access peak. The undefined peak occurs close to the cache-hit peak but is clearly separated from the DRAM-access peak. This behavior suggests that the eviction strategy on the Raspberry Pi 5 is only partially effective, leaving some cache lines in L2 or L3 rather than fully evicting them to DRAM. In contrast, on the Raspberry Pi 4, eviction works as intended, showing a clear separation between cache accesses at approximately 120 CPU cycles and memory accesses at around 325 CPU cycles, with a threshold of roughly 215 CPU cycles. The accesses in the undefined peak are most likely cache hits served from L2 or L3. As a result, some accesses counted as misses in *Flush+Reload* are in fact served from lower-level caches, which explains the observed third peak.

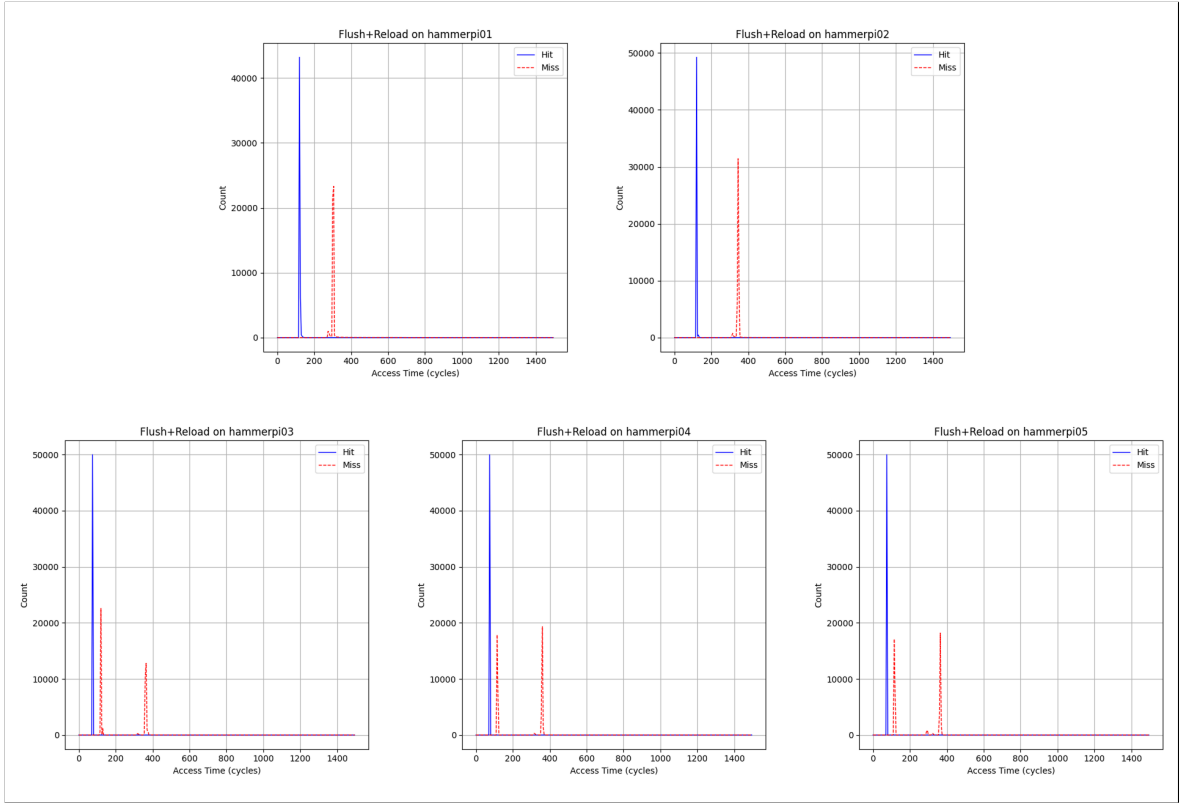


Figure 7: Results of *Flush+Reload*

Flush+Reload and *Prime+Probe* generate a significant number of cache references and cache misses, making them detectable via hardware performance counters [62]. The ***Flush+Flush*** attack addresses this limitation by relying on the execution time of the flush instruction, which differs depending on whether the targeted data is cached or not [22]. The attack setup is similar to *Flush+Reload*. A shared or binary object file is mapped into the attacker’s address

space, an address is flushed from the cache, and the victim program is scheduled. Yet, after the victim executes, the attacker flushes the address again instead of reloading it. This second flush does not generate cache misses, unlike *Flush+Reload* or *Prime+Probe*.

Gruss et al. [22] demonstrated the effectiveness of *Flush+Flush* by constructing a covert channel with a transmission rate nearly seven times faster than previously published channels. Since *Flush+Flush* does not perform any memory accesses, it causes no cache misses at all, and the number of cache hits is reduced to a minimum due to the continuous cache flushes as seen in [22] or on all of our test devices [20].

The ***Evict+Time*** attack measures how evicting a specific cache set affects the execution time [62]. Initially, the execution time of the victim is measured. Subsequently, a specific cache set is evicted, and the victim program is measured again to observe changes in execution time.

The results can be taken from fig. 8. On the one hand, the Raspberry Pi 4, experimental results show a clear separation between cache-resident execution at approximately 175 CPU cycles and memory-resident execution at around 349 CPU cycles, resulting in a meaningful threshold of about 259 CPU cycles. On the other hand, the Raspberry Pi 5's cache and memory access times nearly coincide at roughly 110 and 115 CPU cycles, respectively, yielding no observable timing gap. The absence of a distinct memory-access peak on the Raspberry Pi 5 indicates that the eviction step fails to force victim data out of the effective cache hierarchy. Due to the non-inclusive and dynamically allocated L3 cache, evicted lines may remain in L2 or be serviced by L3, preventing access from transitioning to DRAM. This behavior suggests that cache-set eviction is ineffective for *Evict+Time* on the Raspberry Pi 5, thereby limiting the applicability of this attack on modern ARM cache hierarchies.

Prime+Probe is a cache side-channel attack introduced by Osvik et al. [176] that allows an attacker to identify which cache sets are used during a victim's computations [62]. The attack operates in a manner similar to *Evict+Time*. In the priming phase, the attacker accesses congruent addresses that map to a target cache set, thereby occupying one or more specific cache sets with memory controlled by the adversary, as seen in fig. 9a. During the victim execution phase, the victim program is scheduled and may fill the cache sets with its own addresses (see fig. 9b). If the program uses addresses that map to the same cache set as the attacker's addresses, it will, therefore, evict addresses that the attacker used to occupy the cache set. In the probing phase, as seen in fig. 9c, the attacker reaccesses the previously used addresses and measures the access times to determine which cache lines remain resident. If an address has been evicted, it must be reloaded from main memory, resulting in a significantly

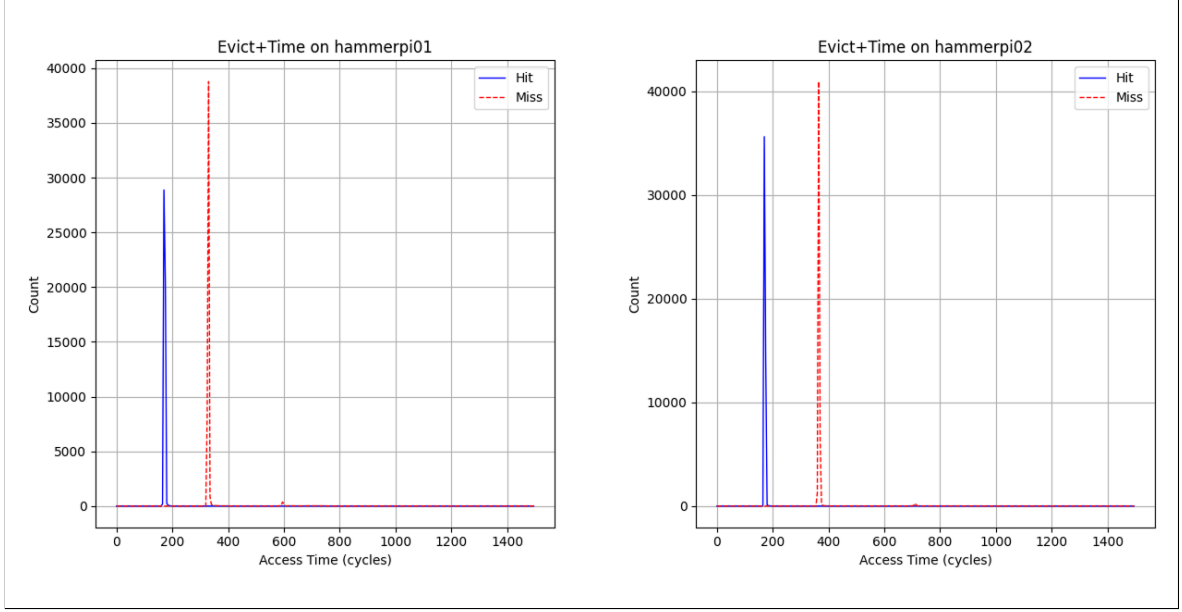


Figure 8: Results of *Evict+Time*

higher access time. Due to the pseudo-random cache replacement policy [62], it is possible that accesses during the probing phase evict previously accessed attacker addresses, which can lead to false positives.

Prime+Probe has been used in several attacks. In 2015, Irazoqui et al. applied *Prime+Probe* to attack the OpenSSL AES implementation in a cloud environment [34]. In the same year, Liu et al. [29] used *Prime+Probe* to construct a cross-core covert channel and to attack ElGamal decryption in GnuPG [29]. Experimental results, as seen in fig. 10, show that *Prime+Probe* fails on the Raspberry Pi 4 when using the selected cache eviction strategy. On the Raspberry Pi 5, observable responses exhibit unusual behavior. We observed instead of two distinct peaks corresponding to cache hits and cache misses, a single broad peak spanning approximately 500 to 1000 CPU cycles. ARM CPUs employ a pseudo-random cache replacement policy to determine which cache line within a set is evicted, which introduces additional noise into time-driven cache attacks [72]. This behavior has been identified as a key factor preventing the successful application of *Prime+Probe* attacks on ARM architectures so far [73, 62].

Evict+Reload was introduced by Gruss et al. in 2015 as a variant of *Flush+Reload* that replaces the flush instruction with cache eviction [25]. While it has no practical use on x86 CPUs due to the availability of the unprivileged `clflush` instruction, it is applicable to ARM CPUs that lack an unprivileged cache flush instruction [82]. Cache eviction in *Evict+Reload* requires filling a cache set with a sufficient number of congruent addresses so that the replacement policy evicts the target address. The number of required addresses and

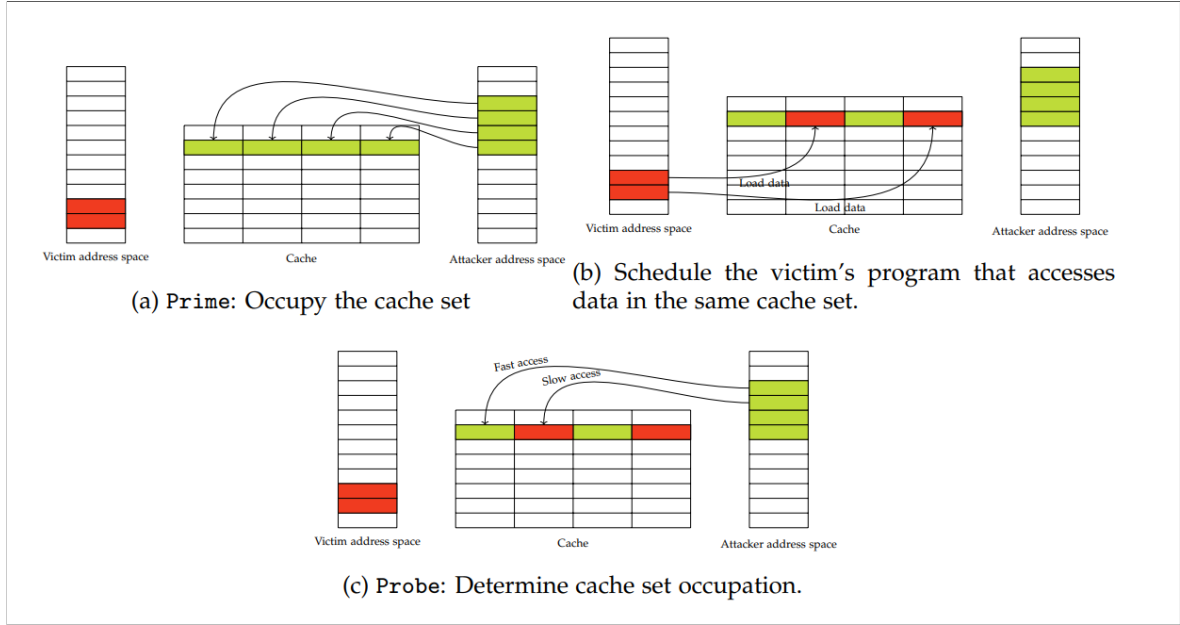


Figure 9: *Prime+Probe* - 4-way (columns) cache with 8 sets (rows) [62]

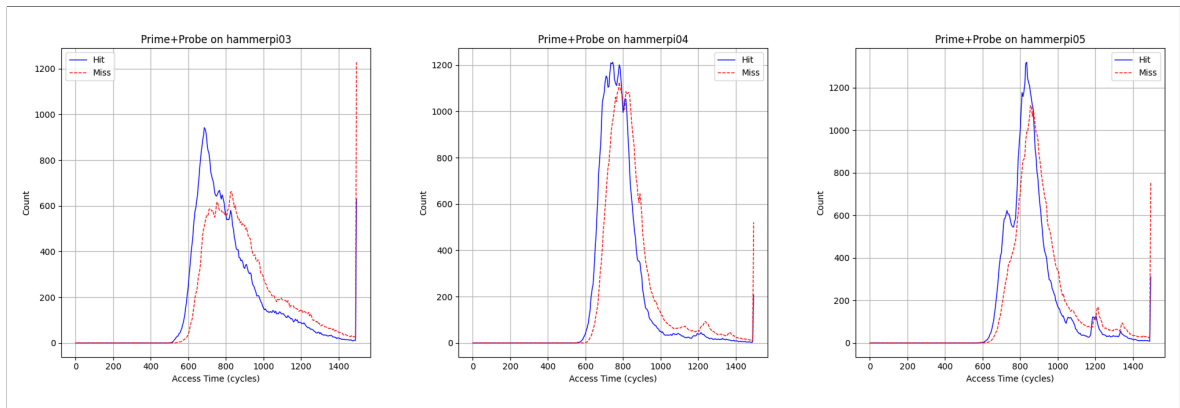


Figure 10: Results of *Prime+Probe*

the access pattern depend on the cache replacement policy, and using too few addresses or accessing them insufficiently results in a low eviction rate [62]. Gruss et al. later demonstrated that effective eviction strategies can even trigger DRAM bitflips, known as Rowhammer, via JavaScript without relying on the privileged `clflush` instruction [21].

The results from fig. 11 obtained with *Evict+Reload* are consistent with those observed using *Flush+Reload*. On the Raspberry Pi 4, *Evict+Reload* works as expected, showing a clear separation between cache and memory accesses at approximately 123 and 323 CPU cycles, respectively, with a threshold of around 222 CPU cycles. On the Raspberry Pi 5, *Evict+Reload* exhibits three distinct access-time peaks corresponding to cache hits at about 75 CPU cycles, an undefined intermediate peak, and DRAM misses at roughly 360 CPU cycles with a threshold near 219 CPU cycles. The intermediate peak is close to the cache-hit peak but clearly separated from DRAM access, indicating that eviction is only partially effective and leaves some cache lines in L2 or L3 rather than fully evicting them to DRAM.

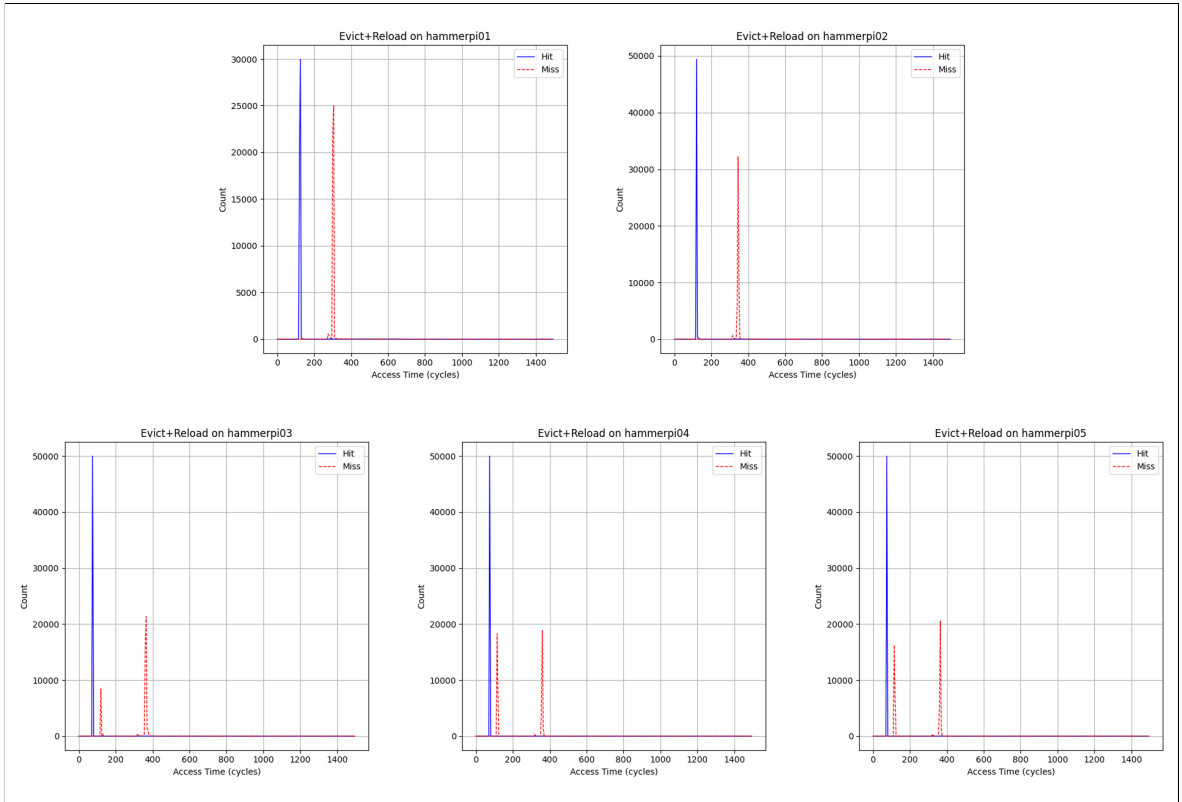


Figure 11: Results of *Evict+Reload*

5.3.2 Feasibility Constraints

The presented findings are applicable to ARMv7- and ARMv8-based systems, including platforms such as the Raspberry Pi 4 and Raspberry Pi 5. Cache eviction can serve as an alter-

native to, or a workaround for, privileged cache maintenance instructions on ARM systems. Some of the presented cache eviction methods can achieve sufficiently high eviction rates comparable to cache maintenance instructions, making side-channel attacks or Rowhammer exploits theoretically feasible. In practice, the execution time of cache eviction is significantly slower than cache flushing, as it requires repeated accesses to a large number of congruent addresses and is affected by noise introduced by pseudo-random cache replacement policies [62]. This increased noise exceeds that of cache flushing, thereby increasing the number of measurements required [59] and further reducing efficiency. Moreover, the most efficient and effective cache eviction strategy must be determined empirically [42], and even the most efficient known strategies remain too slow to reliably trigger the Rowhammer bug on ARM architectures [78].

5.4 Flush Cache

Flushing the cache explicitly invalidates a specific cache line so that the next memory access must be served from DRAM rather than from a cache level. The primary purpose of this operation is to force direct DRAM activation instead of cache hits, thereby guaranteeing a cache miss on every hammer iteration. As a result, very high row-activation rates can be achieved, which are essential for effective Rowhammer attacks. On x86 architectures, this behavior is implemented via the `clflush` instruction [87], whereas ARM-based systems rely on cache flush maintenance instructions, such as `DC CIVAC`, `DC CVAC`, `DC ZVA`, and related operations, to achieve comparable effects [94].

In ARM terminology, the word *flush* is often used in descriptions of clean and invalidate operations, although the architecture specification generally uses only the terms *clean* and *invalidate* [62].

Invalidating a cache or cache line means clearing it of data by clearing the valid bit of one or more cache lines. The cache must always be invalidated after reset because its contents are undefined. If the cache contains dirty data, simple invalidation is generally incorrect, since updated data from write-back cacheable regions would be lost [11].

Cleaning a cache or cache line refers to writing the contents of dirty cache lines back to main memory and clearing the dirty bits, which makes the contents of the cache and main memory coherent with each other [11, 62]. This operation is only applicable to data caches that use a write-back policy. Cache clean and invalidate operations can be performed by cache set or way, or by virtual address.

The ***Point of Coherency (PoC)*** is defined as the point at which all blocks that can access memory, such as processor cores or DMA engines, are guaranteed to observe the same copy of

a given address [11]. Commonly, this point corresponds to the main external system memory, as illustrated in fig. 12b.

Closely related is the *Point of Unification (PoU)*, which denotes the point in the memory hierarchy at which the instruction cache, data cache, and translation table walks of a core are guaranteed to observe the same copy of a memory location [11, 62]. In a system with Harvard L1 caches and a Translation Lookaside Buffer (TLB) for caching translation table entries, a unified L2 cache represents the Point of Unification [11]. If no external cache is present, the main memory itself serves as the PoU [62], as also shown in fig. 12a.

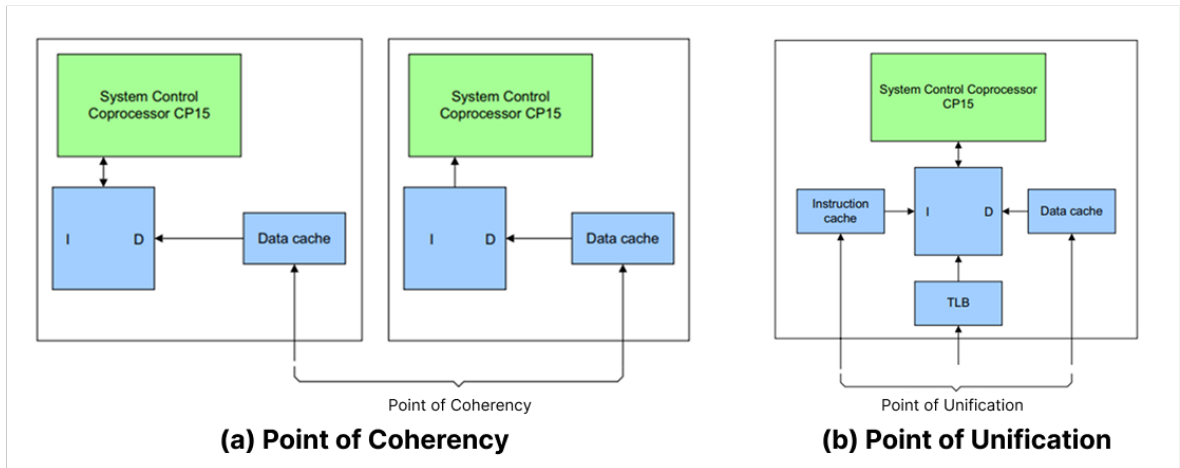


Figure 12: Point of Coherency and of Unification [11]

5.4.1 Instruction Types

The original Rowhammer study by Kim et al. [87] employed the `clflush` instruction to force cache lines out of the cache hierarchy. `clflush` ensures that each memory access reaches physical DRAM, thereby enabling the high-frequency row activations required for Rowhammer. However, Raspberry Pi devices use ARM CPUs, which do not provide `clflush` or a direct equivalent. Instead, these systems rely on ARM cache maintenance instructions to control the cache state. Although architecturally different, these instructions fulfill the same fundamental requirement by bypassing or invalidating caches so that memory accesses are actually served from DRAM. ARM cache maintenance instructions are designed to clean, invalidate, or clean and invalidate specific cache lines or entire cache structures. In the context of Rowhammer attacks on ARM, these instructions are used to invalidate targeted cache lines so that subsequent loads cause DRAM accesses, to clean dirty lines when required so that writes propagate to memory, and to maintain a stable state in which hammering produces real row activations rather than being absorbed by the cache hierarchy. Overall, they are used to force memory accesses to bypass or invalidate cache lines, thereby enabling direct DRAM

access.

On Intel x86 systems, the `clflush` instruction invalidates the cache line containing the specified address from all levels of the cache hierarchy, including both data and instruction caches [62]. If a cache line at any level of the hierarchy is dirty, meaning that it contains modified data [30], it is written back to memory before being invalidated [40, 62]. The `clflush` instruction is not privileged and is therefore available in userspace [40, 62], which makes it particularly suitable for userspace Rowhammer implementations.

On ARM architectures, the `DC CIVAC` instruction performs a clean and invalidate operation on a data cache line by virtual address to the Point of Coherency. If access from Exception Level 0 (EL0) is enabled, executing this instruction at EL0 may generate a permission fault. Its execution may also require a translation from virtual to physical address, and it is defined as a 64-bit system instruction [12, 11]. Similarly, the `DC CVAC` instruction cleans a data cache line by address to the Point of Coherency. It is also a 64-bit system instruction and may generate a permission fault when executed at EL0 if such access is enabled, and it may require address translation [13, 11]. The `DC ZVA` instruction was introduced to efficiently zero out a block of memory [94]. It zeroes a naturally aligned block of N bytes, and is likewise a 64-bit system instruction [14, 11]. The ability to preload the data cache with zero values using `DC ZVA` is new in ARMv8-A. Cache line zeroing behaves similarly to a prefetch by hinting that certain addresses are likely to be used in the future. Nonetheless, zeroing can be significantly faster because it does not require waiting for external memory accesses to complete [94, 86].

5.4.2 Cache Behavior

Table 2 presents cache bypass measurements obtained under different execution modes and measurement units across several Raspberry Pi platforms. The table reports CPU cycle counts for baseline measurements in kernel and user space, as well as measurements obtained using a kernel module and explicit userspace cache-bypass mechanisms using `DC CIVAC` und `STR`. In addition, POSIX functions measure userspace timings in nanoseconds. The userspace

Mode	Unit	RPi4 4 GB	RPi4 8 GB	RPi5 4 GB	RPi5 8 GB	RPi5 16 GB
kernel baseline	Cycles	78.71	78.16	-	-	-
kernel module	Cycles	416.83	468.56	-	-	-
user baseline	Cycles	80.04	88.76	43.19	43.21	43.21
userspace	Cycles	431.76	581.20	489.60	483.14	495.90
userspace	ns	341.92	334.51	263.49	248.34	249.67

Table 2: Variety of cache bypass measurements in user- and kernelspace

measurements demonstrate a significant increase in CPU cycles compared to the respective

baselines. On the RPi4, cache-bypassing measurements in userspace range between 431 and 581 cycles, compared to a baseline of approximately 84 cycles. On the RPi5, userspace cache-bypassing measurements range between 416 and 468 cycles, while the baseline remains at approximately 43 cycles. This clearly indicates that bypassing the cache substantially increases memory access latency.

Kernel module measurements confirm a similar trend for the RPi4. Here, cache-bypassing measurements range from 432 to 581 CPU cycles, compared to a baseline of approximately 78 cycles. For the RPi5, kernel module measurements could not be interpreted due to the Zero Latency Readings Problem described in [46], which prevents meaningful analysis of the collected data.

To statistically evaluate the observed differences, a *Welch t-test* was conducted, yielding a test statistic of $t = -67.70$. The *Welch t-test* assesses whether the means of two independent samples differ significantly without assuming equal variances. The result indicates a statistically significant difference between baseline measurements without cache flushing and measurements obtained using DC CIVAC + STR. In addition, *Cohen's d* was calculated as $d = -3.03$. *Cohen's d* is a standardized measure of effect size that expresses the magnitude of the difference between two means relative to their pooled standard deviation. The obtained value represents a very large effect size, confirming a strong impact of the cache flush operation on memory access latency.

Overall, the results show that the applied cache-bypass mode effectively bypasses the cache hierarchy, leading to an approximately four- to six-fold increase in observed latency on the RPi4 and an approximately five- to six-fold increase on the RPi5. The applied statistical tests support the conclusion that this increase is highly significant and cannot be attributed to random variation. Consequently, DC CIVAC + STR can be considered a robust cache flush operation for inducing measurable memory access delays in both userspace and kernel space, with results that are consistent across multiple devices and measurement interfaces.

6 Address Mapping Reconstruction

Even after eliminating cache effects, a fundamental challenge in memory access remains: the fixed mapping between physical addresses and the underlying DRAM layout. It is insufficient to merely perform DRAM accesses. Rather, precise access to specific DRAM rows is required to induce bitflips in adjacent victim rows, denoted as "*C2. Address Mapping Reconstruction*". The physical address is composed of bits that are partitioned into rank, channel, bank, row, and column, which collectively determine the location where the data is stored [16]. Notably, just like on Intel systems, details of this mapping are not publicly disclosed, unlike the mappings for older CPU models from AMD after they ceased to publish this information with the beginning of microarchitecture 17h [2, 3, 55]. Knowledge of the DRAM address mapping enables a wide range of applications, including application-aware memory channel partitioning, adapted page sizes for improved row buffer utilization, efficient use of emerging hybrid memory technologies, evaluation of the effects of unreliable memory, and DRAM layout-aware memory allocators [16].

When the DRAM mapping is known, memory systems become more susceptible to attacks, leading to higher pressure and an increased likelihood of bitflips. The addressing scheme can vary for each system, as it depends on multiple factors such as the processor model, DIMM population on the motherboard, and BIOS settings, according to [16]. Therefore, it is necessary to reverse-engineer the DRAM address mapping for all test devices. This process leverages the fact that DRAM consistently follows a hierarchical organization, as described in section 2. As a consequence of this structure, memory access latencies vary depending on the physical location of data within the hardware [59].

6.1 Methods for Inferring DRAM Organization

Yun et al. [38] in 2014, uses a dedicated microbenchmark that traverses a linked list over the physical address space to enforce constant DRAM access while controlling bank usage.

Pessl et al. [67] in 2016, exploits the row-buffer timing side channel, where row conflicts cause increased memory access latency. This approach enables the construction of address sets that share channel, rank, and bank properties by performing exhaustive testing of linear addressing functions with an increasing number of coefficients.

Helm et al. [16] in 2020, extends [67] by integrating performance counters, which identifies row hits and misses more reliable during the reverse-engineering process.

Wang et al. [61] in 2020, builds upon [67] by incorporating detailed knowledge of specific DRAM chip geometries and processor microarchitecture, thereby reducing the search space

of the exponential-time [9]. For example, [5] applies this method.

Xiao et al. [88] in 2016, proposes a graph-based approach to infer physical-to-DRAM mappings. It relies on timing differences caused by row conflicts and, similar to [67], primarily aimed at enabling Rowhammer attacks across virtual machines.

Common characteristic approaches include the clustering of addresses into sets that experience row conflicts or row hits, indicating placement within the same bank [6]. These sets are derived by determining which combinations of address bits identify them, often expressed as linear XOR combinations. This approach underlies several attacks and tools, including [82, 6, 62].

Plin et al. [9] 2025, reformulates the identification of DRAM parity masks as a linear algebra problem. Instead of relying on exhaustive search, it focuses on solving the mapping directly through algebraic reasoning.

6.1.1 Knock-Knock Approach

For inferring physical-to-DRAM mappings employs the algebraic framework proposed in [9] as described in the Methods section. This approach formulates parity mask identification as a linear algebra problem. By computing the nullspace of conflicting address sets, it directly derives bank and row functions without iterative function validation. We quickly realized that this approach is not applicable to our systems because, as described in [9], a closed page policy prevents us from obtaining any part of the mapping. Given this constraint, we examined a closed-page policy from a theoretical standpoint. Prior work has shown that the memory controller’s page policy significantly affects Rowhammer behavior and that one-location hammering remains feasible even under a closed-page policy [24, 64]. These findings indicate that triggering Rowhammer does not fundamentally depend on an open-page policy or precise knowledge of the physical DRAM address mapping. For the evaluation, we focused on testing one-location hammering using the [44] tool, which we have ported onto our ARM-based platform. Despite multiple measurement runs exceeding 20 hours in total, we did not observe any bitflips. The most plausible explanations are the presence of TRR or that the deployed DRAM modules are simply not susceptible to Rowhammer-induced disturbances.

6.1.2 Row Hit, Row Conflict Distinction

In the context of reverse engineering DRAM address mappings, it is essential to distinguish between Row Hits and Row Conflicts, as access latency provides a direct reflection of the internal organization of banks and rows. Accurate classification of these events enables the inference of the underlying memory structure and is therefore a critical step in memory

analysis.

This approach builds upon observations from prior work [67, 43, 59]. In particular, three primary cases are considered:

Bank Miss (see fig. 13a): the addresses reside in different banks and access times remain low, analogous to a Row Hit but across banks.

Row Hit (see fig. 13b): An access to a memory address whose corresponding row is already active in the row buffer. Both of these addresses belong to the same bank and the same row, resulting in low access times.

Row Conflict (see fig. 13c): An access targets an address within the same bank while a different row is active in the row buffer, resulting in the need to close the previously opened row and open the new one, producing higher access times.

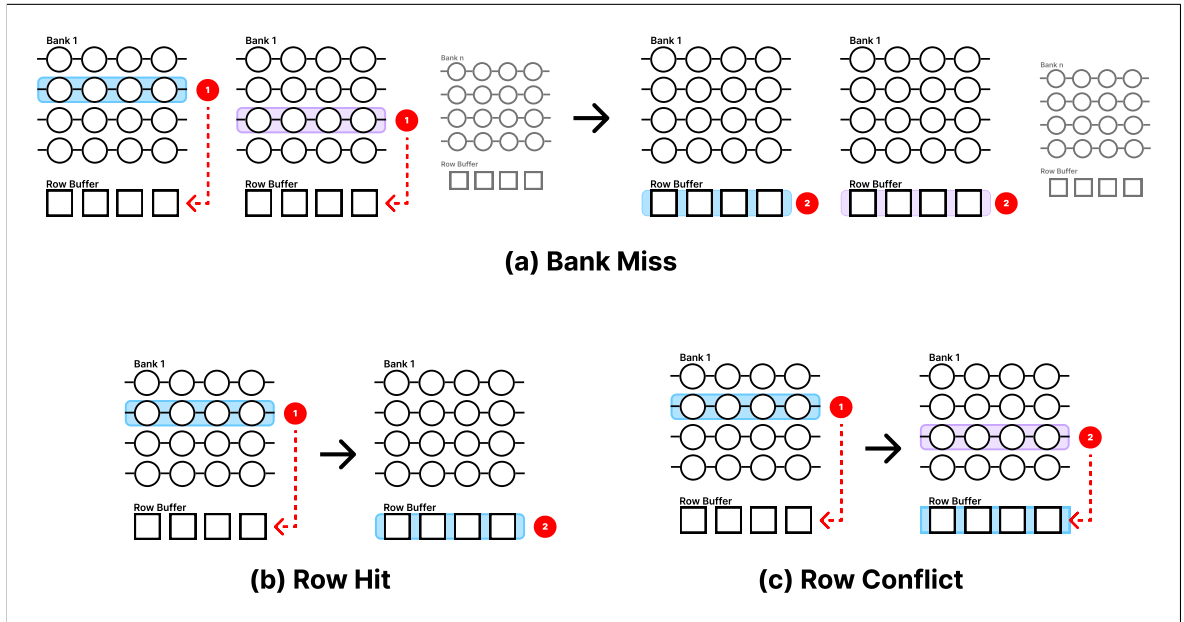


Figure 13: Types of access patterns in DRAM

Based on these scenarios, it is assumed that two distinct peaks will appear in access time distributions: the first peak, corresponding to low-latency events, combines bank misses and row hits, whereas the second peak, associated with higher latencies, represents row conflicts. We measured memory access latencies to reliably distinguish Row Hits from Row Conflicts while isolating DRAM from cache effects, using repeated accesses in combination with cache flushes. Latency histograms constructed from multiple measurement runs, allow identification of distinct peaks. From these histograms, we determine a threshold to separate Row Hits from Row Conflicts, which is subsequently validated using statistical measures such as mean, variance, confidence intervals, and t-tests.

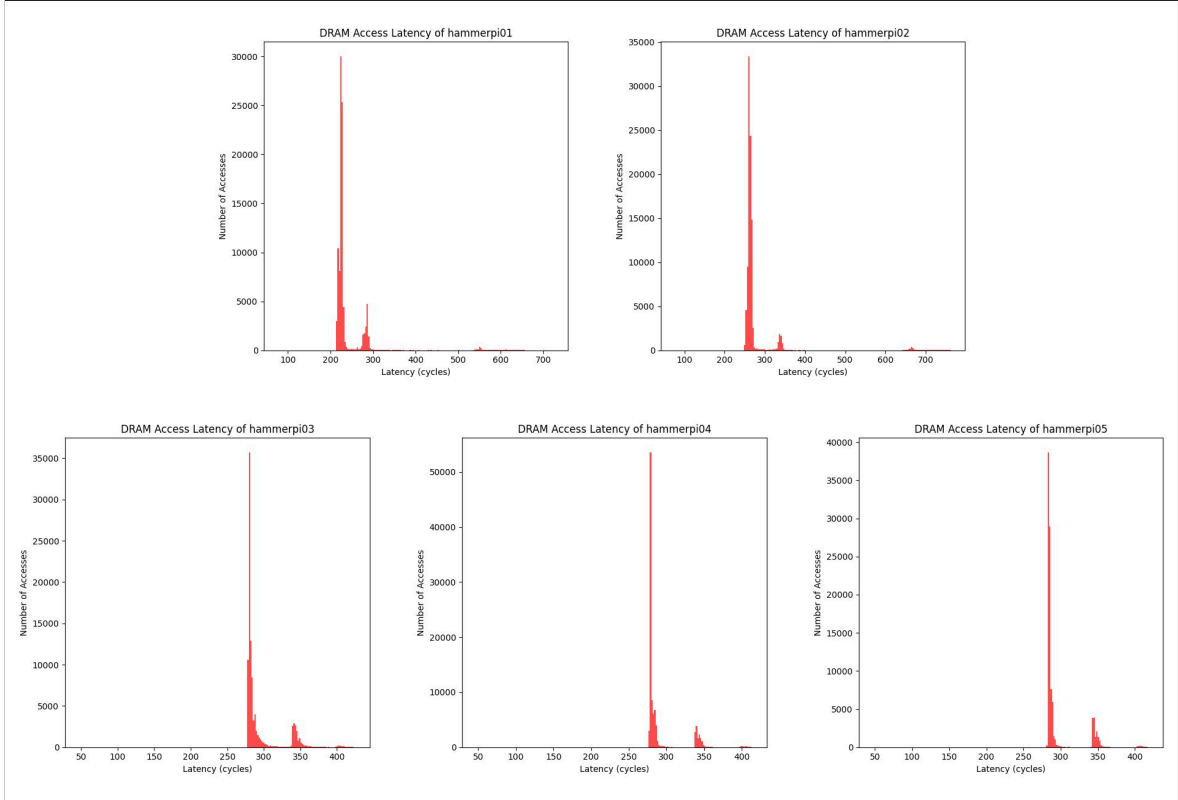


Figure 14: Row hit and -conflict histograms

Figure 14 illustrates the resulting histograms across all test devices. On the Raspberry Pi 4, We observe Row Hits in the range of approximately 210-260 CPU cycles, while Row Conflicts occur shortly before or after 300 CPU cycles, demonstrating a clear separation. For the Raspberry Pi 5, Row Hits occur at approximately 275 CPU cycles and Row Conflicts at 340 CPU cycles, showing a comparable separation but with an approximate 60 CPU cycle shift relative to the predecessor model, the Raspberry Pi 4.

The experimental setup for statistical verification involved pinning all test devices to a single core and limiting memory usage to 20-25 % of total system memory. Across devices, statistical analysis revealed varying degrees of robustness in distinguishing Row Hits from Row Conflicts. Raspberry Pi 4 Devices exhibited the most robust separation, with mean differences of around 23-28 CPU cycles, low variance for hits, well-separated 95 % confidence intervals, and highly significant t-test values of 5.67 and 4.20. Thresholds were clearly defined. One of the Raspberry Pi 5 Devices showed moderate distinction, with a mean separation of 11 CPU cycles, partially overlapping confidence intervals, moderate variance, and a significant t-test value of 3.39, indicating a usable but less robust threshold. In contrast, two out of three Raspberry Pi 5 Devices exhibited less reliable separation, with small mean differences of around 10 CPU cycles to around 7.5 CPU cycles, overlapping confidence intervals, higher

variance, and marginally significant t-tests of 2.70 and 2.28, suggesting sensitivity to noise. Across all devices, Row Hits consistently displayed lower variance than Row Conflicts, reflecting more stable access times. Threshold reliability was highest for the Raspberry Pi 4 Model of 8 GB RAM with a peak separation ratio of 2.00 and a confidence of 0.60, whereas all other test devices exhibited lower ratios of around 1.04 to 1.12 and confidence scores of around 0.45, rendering their thresholds usable but less robust.

6.1.3 DRAMA Approach

The DRAMA approach classifies physical addresses into DRAM sets by exploiting access-time variations caused by row hits, row conflicts, and bank conflicts. Based on these timing differences, the approach derives the DRAM address mapping by identifying XOR combinations of physical address bits that consistently distinguish between the observed sets. Applying this methodology results in the identification of multiple candidate functions, where the frequency of these combinations provides insight into the underlying DRAM organization.

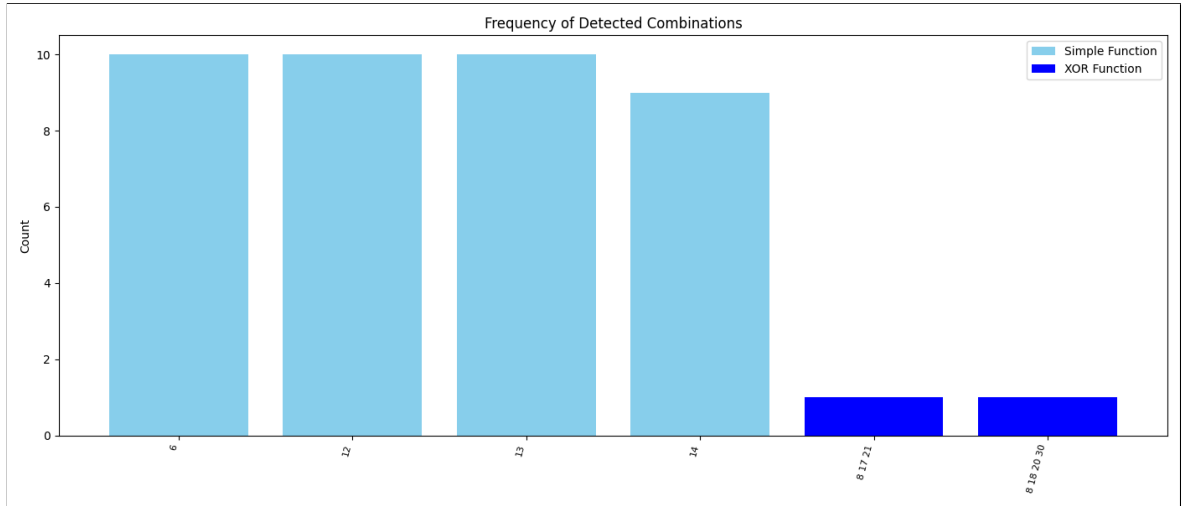


Figure 15: Frequency of detected combinations for Raspberry Pi’s 4

In this work, not only the original DRAMA ARM port without Hugepages was used, but also the TRRespass DRAMA ARM port, which supports Hugepages. Figure 15 and fig. 16 depict the distribution of detected XOR combinations with the original DRAMA port for the Raspberry Pi 4 and Raspberry Pi 5 devices, respectively. In some cases, the analysis yields multiple candidate functions, including rare additional simple bank bits and, in certain instances, more complex XOR combinations. We observe these occurrences more often on Raspberry Pi 5 devices than on Raspberry Pi 4 devices. Prior work on reverse-engineering DRAM address mappings consistently reports that ARM platforms rely exclusively on simple XOR functions, which are typically composed of one or two address bits [67, 5]. Consequently,

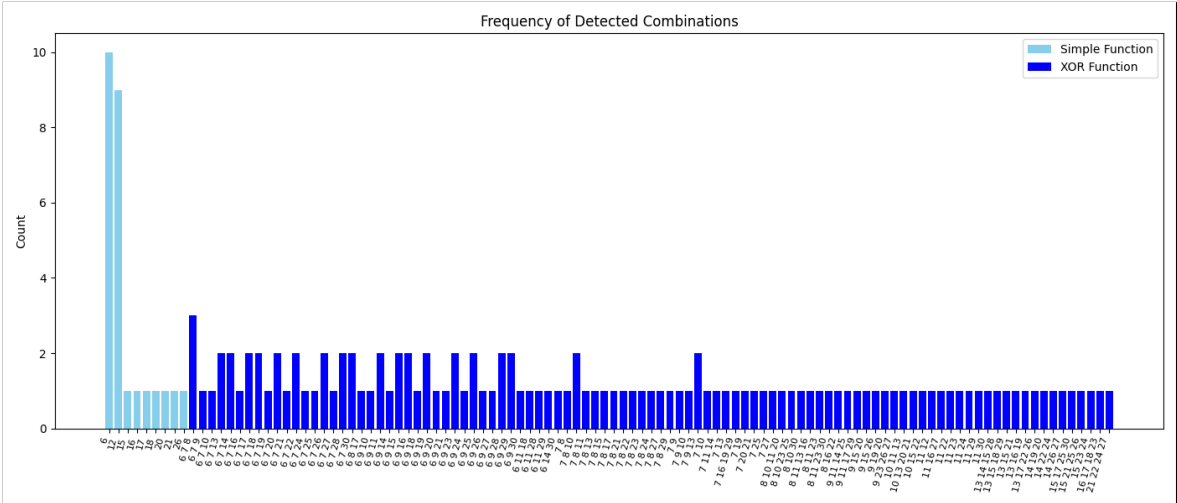


Figure 16: Frequency of detected combinations for Raspberry Pi's 5

this work assumes that the sporadically observed more complex XOR combinations represent false positives, particularly since they occur infrequently and do not exhibit consistency across multiple runs.

When compared to DRAM address mappings reported in related studies, both of our ported ARM DRAMA mappings derived in this work closely resemble those described in [58] and [5], as summarized in table 3.

Device	Mappings	Bank Functions
Raspberry Pi 4B	Memory Aware DOS [58]	(11), (12), (13), (14)
Raspberry Pi 4B	DRAM MaUT [5]	(12), (13), (14), (11 XOR 12)
Raspberry Pi 3B+	Flipping Bits Like a Pro [4]	(13 XOR 14), (14), (15)
Exynos 7420	DRAMA [67]	(14), (15), (16)
Raspberry Pi's 4	Original DRAMA ARM Port [46]	(6), (12), (13), (14)
Raspberry Pi's 5	Original DRAMA ARM Port [46]	(6), (12)
Raspberry Pi's 4 and 5	TRRespass DRAMA ARM Port [46]	(12), (13), (14)

Table 3: Mapping comparison with prior work

6.2 Verification

For each test device, a DRAM address mapping has been derived. Whereas, the completeness and correctness of the recovered mappings cannot be guaranteed. To assess the validity of the inferred DRAM addressing functions, several verification approaches can be employed.

One method involves comparing the inferred functions against results obtained via **physical probing**, as reported in prior work [67]. In general, embedded systems are characterized by small chips and multi-layered circuit boards [59]. On Raspberry Pi devices, direct hardware-

level verification is not feasible, as the DRAM is soldered to the board and therefore inaccessible.

Another verification approach is to use **Rowhammer** experiments to validate the addressing functions. If the inferred functions are correct, their application should increase the bitflip rate proportionally to the number of identified sets [67]. Nevertheless, this method is often impractical due to potentially long execution times, the presence of mitigation mechanisms, or limited susceptibility of the system to Rowhammer. Even on systems previously identified as vulnerable, the distribution and occurrence of vulnerable memory regions vary across DRAM chips, and not every device of the same model exhibits Rowhammer bitflips [82, 62].

A purely **software-based** evaluation constitutes an alternative approach, which verifies timing differences across a larger set of addresses [67]. This method is feasible and allows validation without hardware access.

The verification can be performed using the DRAMA methodology [67, 43]. In this approach, we measured memory access latencies and constructed histograms to distinguish row hits from row conflicts. Address pairs predicted to map to the same DRAM bank and row, denoted as row hits, or to the same bank but different rows, indicates as row conflicts, are identified. After flushing the relevant cache lines to minimize caching effects, we measured access times. Timing histograms are then used to calibrate a threshold separating row hits from row conflicts. The DRAM mapping is validated by comparing the measured latencies with the predicted relationships, as described in the previous section.

The verification assumes that addresses classified as same set and same row indeed result in row hits, and that addresses classified as same set and different row produce row conflicts. If the mapping is correct, two clearly separable timing distributions are expected. The hit histogram should show a dominant peak at low CPU cycle counts, while the miss histogram should exhibit a dominant peak at significantly higher CPU cycle counts. Minimal overlap between hit and conflict histograms is anticipated, and a stable, clear timing gap should exist. This scenario is recognizable when the printed histograms display a bimodal pattern, rows labeled as hits are consistently faster than conflicts, and repeated measurements produce similar histograms and thresholds.

Verification of the DRAM mapping was conducted on multiple Raspberry Pi devices using averages of several measurements (see fig. 17). On Raspberry Pi 4 devices, with the DRAM mapping 6, 12, 13, 14, hammerpi produced distinct peaks corresponding to row hits and row conflicts, although the observed CPU cycles were relatively low. This pattern was consistent across repeated measurements, confirming the presence of measurable row hit and row conflict peaks. In contrast, Raspberry Pi 5 devices, with the DRAM mapping 6, 12, did not yield valid results, as no reliable peaks could be observed. Consequently, while the DRAM

mapping could be validated on Raspberry Pi 4 devices, the verification failed on Raspberry Pi 5 devices, and the underlying cause of this discrepancy remains unclear.

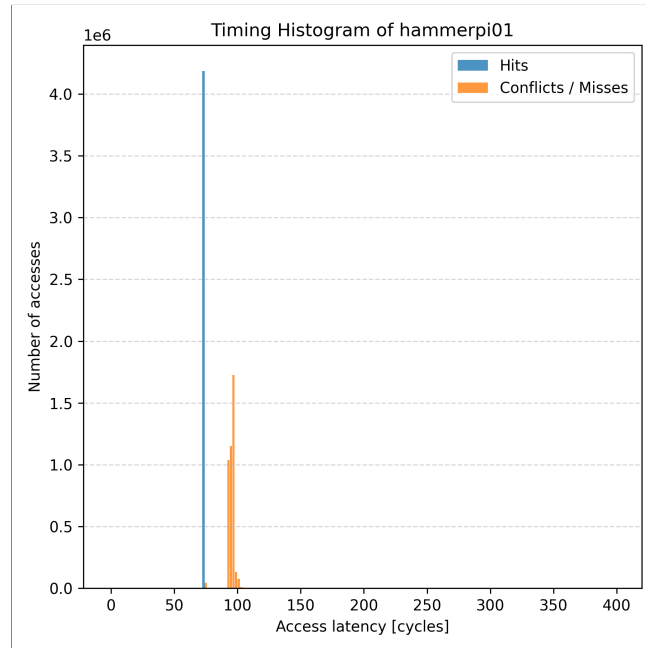


Figure 17: Timing Histogram of Raspberry Pi 4 mapping

7 Outpacing DRAM Refresh

To achieve "*C3. Outpacing DRAM Refresh*", memory accesses must be issued at a rate that exceeds the DRAM refresh behavior, ensuring sufficiently frequent activation's of the same DRAM rows. This section therefore analyzes access and refresh characteristics, examines the impact of existing mitigation mechanisms and the extent to which they must be bypassed to support effective hammering on ARM-based systems such as the Raspberry Pi devices.

7.1 Mitigations

The Rowhammer vulnerability has been demonstrated in a wide range of environments, including sandboxed systems, native platforms, virtual machines, JavaScript contexts, mobile devices, and even across networks [6]. Mitigation strategies aim to reduce both the likelihood and the impact of Rowhammer-induced disturbance errors by altering system behavior, memory management policies, or DRAM operation. Numerous mitigation techniques have been proposed to address the Rowhammer problem in general [94]. These approaches can be broadly classified into two categories: software-based protections and hardware-based protections [85, 82].

7.1.1 Software-based Defenses

Software-based defenses against Rowhammer attacks are often complex, costly, and may remain incomplete [70]. These approaches can be further divided into prevention or restriction techniques and detection strategies.

Prevention and restriction approaches aim to constrain or disable memory access patterns and instructions that can lead to high activation rates, thereby preventing Rowhammer conditions from occurring. Examples include *MASCAT* [33], which performs static analysis of binaries to detect suspicious instruction sequences, though this approach faces open problems such as false positives. *Instruction blacklisting* restricts access to specific commands, such as cache flush instructions, to limit potential exploitation. Additionally, *B-CATT* disables vulnerable physical memory or DRAM regions to prevent Rowhammer-induced errors [31, 39].

Detection approaches focus on identifying behaviors indicative of Rowhammer activity. This includes monitoring abnormal activation frequencies or microarchitectural patterns so that countermeasures can be triggered before bitflips occur. Rowhammer attacks are characterized by a significantly higher number of cache misses and hits compared to other attack types.

Detection systems also aim to identify hammering patterns, frequent row activations, high cache-miss rates (as in *ANVIL* [93]), and other suspicious instruction sequences.

7.1.2 Hardware-based Defenses

The root cause of Rowhammer stems from disturbance errors arising from physical interactions between multiple cells in highly scaled DRAM. Increased integration density and smaller cell geometries reduce noise margins and exacerbate charge leakage effects. A straightforward hardware-based remedy would involve reducing DRAM cell density or increasing the physical cell size. Even so, this would require higher voltages and larger charge reservoirs, resulting in unacceptable power and performance costs. Lowering integration density to mitigate interference also significantly reduces memory capacity, creating a fundamental trade-off between security and storage density. Since commercial DRAM must maximize usable capacity, the underlying physical disturbance mechanism cannot be entirely eliminated in current technology nodes. Moreover, hardware-based approaches may introduce additional latency, increase energy consumption, or raise implementation complexity.

Error-Correcting Code (ECC) can correct single-bit errors using stored parity information [65, 23, 51]. However, ECC can only detect and correct single-bitflips and is insufficient for addressing multi-bitflips, which have become increasingly common [94].

Increasing the DRAM refresh rate represents another mitigation strategy. DRAM cells are volatile and require refreshing at standard intervals of 32 ms to 64 ms [94]. The simplest approach is to increase the refresh rate by two to four times [65, 23], which reduces charge leakage exposure. Nonetheless, higher refresh frequencies delay data access, negatively impacting system performance [23]. While this strategy makes Rowhammer attacks more difficult, it is often unacceptable due to its detrimental effects on performance and energy efficiency [94].

TRR detects frequently activated rows and refreshes adjacent rows to prevent disturbance errors [85, 70]. Nonetheless, *TRR* is vendor-specific and non-mandatory, and it is vulnerable to bypasses such as *TRRespass* [70, 84] and *Halfdouble* [6, 45] due to imperfect identification of victim rows.

7.1.3 Constraints

Several limitations affect mitigation strategies across both software and hardware approaches. Certain hammering techniques can avoid cache misses, rendering detection ineffective. Detection heuristics frequently generate high false-positive rates in real workloads. Mitigations may restrict system functionality or break compatibility with existing software. Single-error

correction remains insufficient, leaving multi-bitflips exploitable. Hardware protections are often manufacturer-dependent (e.g., *TRR*) and may degrade performance, increase energy consumption, or be difficult to deploy on existing platforms, such as ARM architectures [94]. Software-based approaches generally provide only partial workarounds and cannot fully overcome the physical limitations of DRAM, often remaining incomplete. Incorrect identification of victim rows can be exploited, as demonstrated by [70, 6].

To date, existing mitigation strategies only reduce the risk of Rowhammer attacks. No solution provides comprehensive protection across all DRAM generations and attack variants. Full protection would require reliably tracking or constraining every row activation at the hardware level, independent of access patterns, instruction choices, or DRAM vendor behavior, to ensure that no combination of activations could bring any victim cell close to its disturbance threshold.

7.2 TRR Bypass

Although the absence of observed bitflips points toward effective mitigation on Raspberry Pi platforms, these devices were produced recently in 2020 and likely deploy an improved mitigation scheme, as noted in [66]. Prior work research [6, 70, 66, 85] has demonstrated that protection mechanisms such as TRR can often be circumvented through carefully engineered access patterns. To assess whether such bypasses are relevant in this context, this section analyzes TRR behavior in detail and evaluates the applicability of TRR-aware fuzzing techniques on Raspberry Pi devices.

One of the most important terms for this is the Maximum Activate Count (MAC), which specifies the limit on the number of consecutive activations a single memory row can sustain within one refresh interval. When the MAC is reached and further accesses exceed this boundary, the device either executes all required refresh commands before allowing additional row activations or enters TRR mode to refresh only the adjacent victim rows [85, 60].

TRR bypass can be attempted using Rowhammer fuzzers, which are designed to generate variations in inputs or execution patterns that may provoke unexpected system behavior [57, 15]. Specifically, in the context of Rowhammer, this approach targets the circumvention of TRRespass. For this purpose, TRRespass is employed, what represents a scalable black-box Rowhammer fuzzer introduced in 2020 by Frigo et al. [70]. It is designed for "TRR-aware" Rowhammer analysis and exploitation, primarily on x86 systems. The tool automatically identifies access patterns capable of bypassing TRR based on two parameters: cardinality and location.

The use of hugepages is critical for TRRespass functionality. In modern processor architec-

tures, the CPU and Operating System (OS) must keep track of mappings between virtual pages and their corresponding physical memory locations. As the number of memory pages increases, locating the corresponding physical memory mappings become more time-consuming. Larger page sizes reduce the number of page entries that need to be traversed [28]. TRRespass relies on hugepages to obtain large, physically contiguous memory regions, which are necessary for reliably mapping virtual to physical addresses and targeting specific DRAM rows. Without hugepages, many hammering strategies either fail to function reliably or do not work at all, as indicated in [70].

For the evaluation of TRRespass, three out of five test devices were successfully used. The remaining two devices, referred to as Raspberry Pi 5 with 4 and 8 GB RAM, encountered an unexpected error during the pre-allocation of a single 1 GB hugepage, which could not be resolved even after rebuilding the kernel multiple times. Due to the underlying smaller hugepage sizes, TRRespass only works suboptimally and less reliably, which is why these results are not discussed. On the devices where hugepages were functional, TRRespass was executed with varying parameters, access patterns, and DRAM mappings, including reverse-engineered mappings as well as those provided by [67, 58, 5]. Across all evaluated devices, we did not observe any effective access pattern capable of inducing bitflips. Even after multiple experiments varying in runtime length of 5 to 10 and in some cases also over 20 hours all tested access patterns, including N-sided, 2-sided, and 3-sided, results in no observable or reproducible bitflips on any of our test devices

These findings suggest that the Raspberry Pi 4 and 5 models are either not vulnerable to Rowhammer attacks or exhibit the same limitations observed in certain LPDDR4x devices studied by [66]. Additional details are discussed in section 9.

8 Related Work

It all began in 2014, when Kim et al. [87] discovered the rowhammer vulnerability in the former DDR3 of an x86 system and talks about their technical details. Seaborn et al. [53] subsequently developed a working probabilistic exploit that use this effect for local privilege escalation on year later. Shortly thereafter, in 2016, Gruss et al. [21] demonstrated a Rowhammer attack in JavaScript that did not rely on the CLFLUSH instruction, instead evicting cache lines from the CPU cache [54]. This work showed that the previously considered essential instruction was no longer required and revealed a wide range of additional exploitation possibilities of the vulnerability. In the same year, Pessl et al. [67] introduced an automated reverse-engineering approach to identify DRAM addressing functions using their tool [43]. Their method leveraged timing-based measurements to systematically reconstruct the mapping of physical addresses to DRAM channels, ranks, and banks, thereby enabling more precise and targeted memory-based attacks. Schwarz et al. [59] extended the theoretical and practical foundations of DRAMA introduced by Pessl et al., adapting the approach for ARM-based systems. Although the original implementation was not publicly available, their work provided the basis for our thesis to port DRAMA to ARM, enabling systematic analysis of DRAM buffer on modern ARM platforms.

Earlier Rowhammer attacks primarily targeted x86 systems, largely because the unprivileged CLFLUSH instruction enables efficient exploitation [87, 86]. As ARM has become the dominant architecture in mobile devices, researchers have attempted to adapt these x86-oriented techniques. However, this translation is challenging, and some have even suggested that Rowhammer attacks on ARM might be infeasible, partly due to the potentially slower memory controllers that could prevent the bug from being triggered [82, 94]. Nevertheless, subsequent studies demonstrated that Rowhammer can indeed be exploited on ARM by identifying alternative methods to bypass these architectural limitations. [82] were the first study of Rowhammer prevalence on ARM-based devices, including Android smartphones and Chromebooks, as it was released from Van der Veen et al. in 2016. They provide a deterministic Rowhammer attack that cannot be mitigated by current defenses by leveraging user-accessible DMA buffers on Android devices, which allow memory accesses to bypass the CPU caches and directly interact with DRAM. Lipp et al. [63] demonstrated in 2016 cache side-channel attacks on ARM based mobile devices without requiring privileged instructions and to replace CLFLUSH, enabling fine-grained timing attacks on ARM. Subsequently, the authors of [62] extended such cache attacks on ARM architectures to practical exploitation scenarios, including keystroke inference, attacks on cryptographic algorithms, and the induction of Rowhammer faults. This methodology serves as a foundation in our thesis, where it is

adopted as one approach to effectively bypass the CPU cache. Zhang et al. [94] validates in 2018 the attack on ARMv8-based single-board computers, namely by Amlogic S905X SBC by reliably trigger bitflips through ARM cache maintenance instructions analogous to the case of x86 [86].

Bekele et al. [86] demonstrated the susceptibility of the Raspberry Pi 3B+ to Rowhammer attacks by leveraging ARM cache maintenance instructions. In addition, they analyzed the impact of DSB on attack effectiveness and examined the likelihood of inducing repeated bitflips at specific memory locations. Plin et al. [9] introduced in 2025 a black-box approach to reverse-engineer DRAM address mappings without relying on proprietary hardware documentation, evaluating their method on a comparable platform, namely the Raspberry Pi 4. Nevertheless, they observed and induced a closed-page policy on this system, influencing the effectiveness of reconstructing the underlying DRAM address mapping dramatically.

Previous studies have demonstrated the susceptibility of Rowhammer on older Raspberry Pi models. Even so, no other embedded systems within the Raspberry Pi series were evaluated. To date, research on Rowhammer has been limited almost exclusively to the Raspberry Pi 4, while the Raspberry Pi 5 remains entirely unexplored. Whereas the Raspberry Pi 4 has only been scarcely investigated and explored to trigger Rowhammer faults with Graphics Processing Unit (GPU)-based memory access patterns [8] in 2025. Consequently, it remains an open question whether modern Raspberry Pi platforms, such as the Raspberry Pi 4 and Raspberry Pi 5, are susceptible to software-induced Rowhammer attacks.

9 Discussion and Limitations

In this work, an issue was observed when using the Knock-Knock tool [10] to reverse engineer the DRAM address mapping. Specifically, the tool reported the failure condition “*Valley between peaks not pronounced enough*”. As a consequence, no part of the mapping could be retrieved from the measurements.

A closely related issue has been reported in prior work. The authors of [9] encountered the same problem on LPDDR-based devices, including the Raspberry Pi 4 with 4 GB of RAM and the Nvidia Jetson Nano. They attributed this behavior to the presence of a closed page policy, as their latency measurements showed only a single distribution rather than clearly separable peaks. Similar behavior was observed on additional platforms, including the Raspberry Pi 4 with both 4 GB and 8 GB of RAM, as well as the newer Raspberry Pi 5 with 4 GB, 8 GB, and 16 GB of RAM, when analyzed using the Knock-Knock tool [10].

Under a closed page policy, the memory controller immediately closes the currently active row after each memory access [56, 9]. In contrast, an open page policy, which is the default in most systems, keeps a DRAM row active in the row buffer until a subsequent access targets a different row, at which point the currently open row must be closed [24, 56].

Based on the measurements conducted in this work, the most likely explanation for the observed behavior is not the presence of a closed page policy, but rather an unclear limitation of the [9] methodology on the evaluated platforms. Although the tool fails with the error message, the measurements demonstrate that row hits and row conflicts can, in principle, be distinguished, indicating that the row buffer is not always closed. This apparent contradiction is further supported by other studies [5, 58], which evaluated the same device, namely the Raspberry Pi 4. Consequently, this limitation does not arise from the DRAM configuration itself, but from the constraints of the [9] methodology. While the underlying memory behavior allows distinguishing row hits from row conflicts, the tool fails to recover the mapping due to its thresholding and peak-detection assumptions.

After successfully porting TRRespass to ARM, extensive testing was performed over several hours using different configurations and multiple runs in an attempt to bypass TRR mechanisms and induce bitflips. Despite these efforts, no effective hammering pattern emerged when using the fuzzer.

Similar limitations have been reported in [66] on some of their tested Micron LPDDR4x devices as they were unable to trigger bitflips. Regardless, the official publication does not specify the exact models or their architectures. Especially newer modules manufactured in 2020 were affected due to improved mitigation mechanisms such as strengthened TRR implementations. As a result, its effectiveness on LPDDR devices was shown to be platform-

dependent and not guaranteed across all chips.

Blacksmith also did not work on these devices out of the box without reverse engineering the TRR behavior. Its default configurations assumed TRR distances, hammering amplitudes, and refresh alignments that did not correspond to the actual TRR behavior of certain LPDDR4x devices. The failure on some LPDDR4x modules was attributed to several factors. First, TRR distances were larger and more irregular than assumed by the default parameter ranges. Second, effective hammering patterns required longer consecutive hammering phases and precise alignment with specific refresh commands. Third, correct refresh alignment was critical but not accounted for in the initial implementation [17]. Finally, the required reverse engineering depended on control over refresh commands, which is generally unavailable outside controlled test platforms.

These findings directly relate to the limitations of the present work. In the evaluation, the authors had full control over the refresh commands on their test devices, which enabled them to reverse engineer the TRR behavior. In contrast, the experimental setup used in this work does not provide access to refresh control, thereby preventing a comparable analysis of TRR mechanisms. Consequently, the lack of refresh awareness and the restricted fuzzing capabilities limit the reliable identification of effective attack patterns.

To make Blacksmith-like approaches work on similar LPDDR devices without refresh control, alternative strategies are needed. These include probabilistic or exhaustive fuzzing with extended parameter ranges to compensate for unknown TRR distances and refresh behavior, as well as accepting longer runtimes and repeated attempts, since effective patterns require alignment with refresh intervals that cannot be directly controlled. However, without explicit control over refresh commands, the probability of accidentally achieving the correct refresh alignment remains small, particularly given larger TRR distances and irregular refresh behavior. Achieving such an alignment would require very long runtimes and many repeated attempts, rendering the approach impractical in most realistic settings.

10 Conclusion

This thesis examined the resilience of contemporary Raspberry Pi platforms against disturbance-based memory faults. In particular, it investigated whether the Raspberry Pi 4 and Raspberry Pi 5 exhibit susceptibility to Rowhammer-induced bitflips when exposed to controlled hammering access patterns. The objective was to determine the extent to which these modern embedded systems remain vulnerable to software-triggered DRAM disturbance errors.

The conducted experiments yielded several central findings. With regard to cache bypass mechanisms, two out of four investigated approaches proved to be feasible. Cache bypass could be achieved either through eviction strategies or by flushing the cache using maintenance instructions. These methods enabled uncached memory accesses required for effective hammering attempts.

Concerning DRAM mapping, the DRAMA tool [43] was successfully ported to the ARM architecture. The adapted implementation provided accurate DRAM address mappings for the respective test platforms.

In terms of outrunning the refresh mechanism, the TRRespass tool [84] was likewise ported successfully to ARM systems. Although the tool operated as intended, it does not identify any effective access patterns for the test devices.

Based on the experimental evaluation, it was concluded that modern Raspberry Pi models, including Raspberry Pi 4 and Raspberry Pi 5 with RAM configurations ranging from 4 GB to 16 GB, did not exhibit susceptibility to Rowhammer under the tested conditions. Nevertheless, certain limitations prevent a definitive conclusion regarding absolute immunity. In particular, the potential applicability of advanced techniques such as Blacksmith [17] and the influence of the DRAM page policy restrict the scope of the conclusion.

The absence of observable bitflips contrasts with prior expectations. Earlier studies indicated that previous models, such as the Raspberry Pi 3, were susceptible to Rowhammer attacks [86], and GPU-based hammering [8] on the Raspberry Pi 4. The lack of observable bitflips in the current study is likely due to active TRR mechanisms, which mitigate disturbance effects and prevent reliable fault induction.

The findings narrow the immediate Rowhammer threat landscape for modern Raspberry Pi platforms. As of the time of this study, experimental results indicate that these devices are effectively resilient to conventional software-induced Rowhammer attacks.

This work provides the first systematic evaluation of Rowhammer susceptibility on the Raspberry Pi 5 across multiple RAM configurations, including 4 GB, 8 GB, and 16 GB variants. It empirically demonstrates that two practical cache-bypass methods are implementable on the Raspberry Pi 4 and Raspberry Pi 5. Furthermore, despite achieving successful cache bypass

and accurate DRAM mapping, no effective hammering patterns could be induced.

The results offer indirect evidence for the effectiveness of TRR as a mitigation mechanism on ARM-based single-board computers. The study extends prior research that focused primarily on the Raspberry Pi 3 and partially on the Raspberry Pi 4. It provides clear experimental evidence that, under the applied conditions, no reproducible bitflips occur.

In addition, the a DRAM Address Function reverse engineer approach using foundation of [67] was successfully ported to ARM systems and specifically adapted for the Raspberry Pi platform. Similarly, a fully ported and published version of the TRRespass tool [84] for ARM systems was developed, thereby enabling further evaluation and experimentation.

Several open questions and potential extensions arise from this work.

First, advanced investigations into TRR bypass techniques should be conducted. This includes integrating and evaluating Blacksmith-style frequency-based hammering. Furthermore, vendor-specific TRR behavior on LPDDR memory should be analyzed to determine mitigation thresholds and activation logic. The development of adaptive hammering techniques aimed at bypassing TRR and generating effective disturbance patterns represents another important research direction.

Second, further analysis of the page policy and memory controller behavior is required. The active DRAM page policy could be characterized using [64]. Additionally, the impact of the page policy on row activation frequency, row buffer behavior, and overall disturbance feasibility should be evaluated. Finally, methods to influence or manipulate page policy behavior should be investigated in order to assess worst-case scenarios.

Appendix A Listings

References

- [1] Ademodi Oluwatosin Abayomi, Ajayi Abayomi Olukayode, and Green Oluwole Olakunle. “An Overview of Cache Memory in Memory Management”. In: (2020). DOI: <https://doi.org/10.11648/j.acis.20200803.11>. (Visited on 11/26/2025).
- [2] AMD. *AMD’s Zen CPU is now called Ryzen, and it might actually challenge Intel*. 2016. URL: <https://arstechnica.com/gadgets/2016/12/amd-zen-performance-details-release-date/> (visited on 02/17/2026).
- [3] AMD. *Developer Central*. 2020. URL: <https://www.amd.com/en/developer.html> (visited on 02/17/2026).
- [4] Anandpreet Kaur, Pravin Srivastav, and Bibhas Ghoshal. “Flipping Bits Like a Pro”. In: (2023). DOI: <https://doi.org/10.1109/LES.2023.3298737>. (Visited on 01/28/2026).
- [5] Anandpreet Kaur, Pravin Srivastav, and Bibhas Ghoshal. “Work-in-Progress: DRAM-MaUT: DRAM Address Mapping Unveiling Tool for ARM Devices”. In: (2022). DOI: <https://doi.org/10.1109/CASES55004.2022.00027>. (Visited on 11/26/2025).
- [6] Andreas Kogler, Jonas Juffinger, Salman Qazi, Yoongu Kim, Moritz Lipp, Nicolas Boichat, Eric Shiu, Mattias Nissler, and Daniel Gruss. “Half-Double: Hammering From the Next Row Over”. In: (2022). URL: <https://www.usenix.org/system/files/sec22-kogler-half-double.pdf> (visited on 11/26/2025).
- [7] Andrew J. Walker, Sungkwon Lee, and Dafna Beery. “On DRAM Rowhammer and the Physics of Insecurity”. In: (2021). DOI: <https://doi.org/10.1109/TED.2021.3060362>. (Visited on 11/26/2025).
- [8] Antoine Plin, and Frédéric Fauberteau. “OpenGL GPU-Based Rowhammer Attack (Work in Progress)”. In: (2025). DOI: <https://doi.org/10.48550/arXiv.2509.19959>. (Visited on 11/26/2025).
- [9] Antoine Plin, Lorenzo Casalino, Thomas Rokicki, and Ruben Salvador. “Knock-Knock: Black-Box, Platform-Agnostic DRAM Address-Mapping Reverse Engineering”. In: (2025). URL: <https://arxiv.org/pdf/2509.19568> (visited on 11/26/2025).
- [10] antpln. *Knock-Knock*. 2025. URL: <https://github.com/antpln/Knock-Knock> (visited on 11/26/2025).
- [11] ARM. *ARM® Cortex™ A Series Version: 4.0 Programmers Guide*. URL: <https://open432.github.io/resources/arm-cortex-a-prog-guide-v4.pdf> (visited on 11/26/2025).
- [12] ARM. *DC CIVAC, Data or unified Cache line Clean and Invalidate by VA to PoC*. URL: <https://developer.arm.com/documentation/111107/2025-09/AArch64-Instructions/DC-CIVAC--Data-or-unified-Cache-line-Clean-and-Invalidate-by-VA-to-PoC?lang=en> (visited on 01/28/2026).
- [13] ARM. *DC CVAC, Data or unified Cache line Clean by VA to PoC*. URL: https://developer.arm.com/documentation/111180/2025-09_ASL1/AArch64-Instructions/DC-CVAC--Data-or-unified-Cache-line-Clean-by-VA-to-PoC (visited on 01/28/2026).

-
- [14] Arm Limited. *DC ZVA, Data Cache Zero by VA*. 2024. URL: <https://df.lth.se/~getz/ARM/SysReg/AArch64-dc-zva.html> (visited on 01/28/2026).
 - [15] Ben Lutkevich. *Fuzz-Testing (Fuzzing)*. 2024. URL: <https://www.computerweekly.com/de/definition/Fuzz-Testing-Fuzzing> (visited on 02/02/2026).
 - [16] Christian Helm, Soramichi Akiyama, and Kenjiro Taura. “Reliable Reverse Engineering of Intel DRAM Addressing Using Performance Counters”. In: (2020). DOI: <https://doi.ieeecomputersociety.org/10.1109/MASCOTS50786.2020.9285962>. (Visited on 11/26/2025).
 - [17] comsec-group. *blacksmith*. 2024. URL: <https://github.com/comsec-group/blacksmith> (visited on 11/26/2025).
 - [18] D. -S. M. Dong-Sun Min, D. -I. S. Dong-II Seo, J. Y. Jehwan You, S. C. Sooin Cho, D. C. Daeje Chin and Y. E. Park. “Wordline coupling noise reduction techniques for scaled DRAMs”. In: (1990). DOI: 10.1109/VLSIC.1990.111105. (Visited on 02/06/2026).
 - [19] Daniel Gruss. *Rowhammer Attacks: An Extended Walkthrough Guide*. 2017. URL: <https://gruss.cc/files/sba.pdf> (visited on 11/26/2025).
 - [20] Daniel Gruss. “Software-based Microarchitectural Attacks”. PhD Thesis. TU Graz, 2017. URL: <https://arxiv.org/pdf/1706.05973> (visited on 02/13/2026).
 - [21] Daniel Gruss, Clémentine Maurice, and Stefan Mangard. “Rowhammer.js: A Remote Software-Induced Fault Attack in JavaScript”. In: (2016). DOI: <https://doi.org/10.48550/arXiv.1507.06955>. (Visited on 11/26/2025).
 - [22] Daniel Gruss, Clémentine Maurice, Klaus Wagner, and Stefan Mangard. “Flush+Flush: A Fast and Stealthy Cache Attack”. In: (2016). DOI: <https://doi.org/10.48550/arXiv.1511.04594>. (Visited on 01/08/2026).
 - [23] Daniel Gruss, Martin Heckel, and Florian Adamsky. *Ten Years of Rowhammer: A Retrospect (and Path to the Future)*. 2024. URL: <https://media.ccc.de/v/38c3-ten-years-of-rowhammer-a-retrospect-and-path-to-the-future#t=1177> (visited on 11/26/2025).
 - [24] Daniel Gruss, Moritz Lipp, Michael Schwarz, Daniel Genkin, Jonas Juffinger, Sioli O’Connell, Wolfgang Schoechl, and Yuval Yarom. “Another Flip in The Wall Of Rowhammer Defense”. In: (2018). DOI: <https://doi.org/10.48550/arXiv.1710.00551>. (Visited on 11/26/2025).
 - [25] Daniel Gruss, Raphael Spreitzer, and Stefan Mangard. “Cache Template Attacks: Automating Attacks on Inclusive Last-Level Caches”. In: (2015). URL: <https://www.usenix.org/system/files/conference/usenixsecurity15/sec15-paper-gruss.pdf> (visited on 01/07/2026).
 - [26] David Gullasch, Endre Bangerter, and Stephan Krenn. “Cache Games – Bringing Access-Based Cache Attacks on AES to Practice”. In: (2011). DOI: 10.1109/SP.2011.22. (Visited on 01/07/2026).
 - [27] Dayeon Kim, Hyungdong Park, Inguk Yeo, Youn Kyu Lee, Youngmin Kim, Hyung-Min Lee, and Kon-Woo Kwon. “Rowhammer Attacks in Dynamic Random-Access Memory and Defense Methods”. In: (2024). DOI: <https://doi.org/10.3390/s24020592>. (Visited on 11/26/2025).

-
- [28] Debian. *Hugepages*. 2023. URL: <https://wiki.debian.org/Hugepages> (visited on 02/02/2026).
 - [29] Fangfei Liu, Yuval Yarom, Qian Ge, Gernot Heiser, and Ruby B. Lee. “Last-Level Cache Side-Channel Attacks are Practical”. In: (2015). DOI: 10.1109/SP.2015.43. (Visited on 01/08/2026).
 - [30] Felix Cloutier. *CLFLUSH - Flush Cache Line*. URL: <https://www.felixcloutier.com/x86/clflush> (visited on 01/28/2026).
 - [31] Ferdinand Brasser, Lucas Davi, David Gens, Christopher Liebchen, and Ahmad-Reza Sadeghi. “Can’t Touch This: Practical and Generic Software-only Defenses Against Rowhammer Attacks”. In: (2016). DOI: <https://doi.org/10.48550/arXiv.1611.08396>. (Visited on 11/26/2025).
 - [32] Florian Adamsky. “Rowhammer on Raspberry Pi’s: Assignment Sheet”. In: (2025). (Visited on 02/04/2026).
 - [33] Gorka Irazoqui, Thomas Eisenbarth, and Berk Sunar. “MASCAT: Preventing Microarchitectural Attacks Before Distribution”. In: (2018). DOI: <https://doi.org/10.1145/3176258.3176316>. (Visited on 11/27/2025).
 - [34] Gorka Irazoqui, Thomas Eisenbarth, and Berk Sunar. “S\$A: A Shared Cache Attack that Works Across Cores and Defies VM Sandboxing – and its Application to AES”. In: (2015). DOI: 10.1109/SP.2015.42. (Visited on 01/08/2026).
 - [35] GScience Direct. *Sense Amplifier*. 2024. URL: <https://www.sciencedirect.com/topics/engineering/sense-amplifier> (visited on 11/26/2025).
 - [36] hammertux. *rowhammering*. 2019. URL: <https://github.com/hammertux/rowhammering> (visited on 11/26/2025).
 - [37] Haocong Luo, İsmail Emir Yüksel, Ataberk Olgun, A. Giray Yaglikci, and Onur Mutlu. “Revisiting DRAM Read Disturbance: Identifying Inconsistencies Between Experimental Characterization and Device-Level Studies”. In: (2025). DOI: <https://doi.org/10.48550/arXiv.2503.16749>. (Visited on 11/26/2025).
 - [38] Heechul Yun, Renato Mancuso, Zheng-Pei Wu, and Rodolfo Pellizzoni. “PALLOC: DRAM Bank-Aware Memory Allocator for Performance Isolation on Multicore Platforms”. In: (2014). DOI: <https://doi.org/10.1109/RTAS.2014.6925999>. (Visited on 11/26/2025).
 - [39] IEEE Symposium on Security and Privacy. *Another Flip in the Wall of Rowhammer Defenses*. 2018. URL: https://www.youtube.com/watch?v=_JPhsqJzV5I&t (visited on 11/26/2025).
 - [40] Intel. *Intel® 64 and IA-32 Architectures Optimization Reference Manual*. 2023. URL: <https://www.intel.com/content/www/us/en/content-details/671488/intel-64-and-ia-32-architectures-optimization-reference-manual-volume-1.html> (visited on 12/03/2025).
 - [41] IONOS Redaktion. *Was ist ein Cache?* 2022. URL: <https://www.ionos.de/digitalguide/hosting/hosting-technik/was-ist-ein-cache/> (visited on 11/26/2025).

-
- [42] isec-tugraz. *armageddon*. 2017. URL: <https://github.com/isec-tugraz/armageddon> (visited on 11/26/2025).
 - [43] isec-tugraz. *drama*. 2017. URL: <https://github.com/isec-tugraz/drama> (visited on 11/26/2025).
 - [44] isec-tugraz. *flipfloyd*. 2018. URL: <https://github.com/isec-tugraz/flipfloyd> (visited on 11/26/2025).
 - [45] isec-tugraz. *halfdouble*. 2022. URL: <https://github.com/isec-tugraz/halfdouble> (visited on 11/26/2025).
 - [46] Jakob Drescher. “Rowhammer on Raspberry Pi’s: Practical Work”. In: (2026). (Visited on 02/24/2026).
 - [47] japanninja74. *ES_Security_rowhammer_rpi3*. 2023. URL: https://github.com/japanninja74/ES_Security_rowhammer_rpi3 (visited on 11/26/2025).
 - [48] JEDEC. *JESD209-4, LPDDR4 Specification*. 2024. URL: <https://www.jedec.org/standards-documents/docs/jesd209-4b> (visited on 11/26/2025).
 - [49] JEDEC. *JESD209-4, LPDDR4X Specification*. 2025. URL: <https://www.jedec.org/standards-documents/docs/jesd209-4-1> (visited on 11/26/2025).
 - [50] Kaveh Razavi, Ben Gras, Erik Bosman, Bart Preneel, Cristiano Giuffrida, and Herbert Bos. “Flip Feng Shui: hammering a needle in the software stack”. In: (2016). URL: https://www.usenix.org/system/files/conference/usenixsecurity16/sec16_paper_razavi.pdf (visited on 02/12/2026).
 - [51] Lucian Cojocar, Kaveh Razavi, Cristiano Giuffrida, and Herbert Bos. “Exploiting Correcting Codes: On the Effectiveness of ECC Memory Against Rowhammer Attacks”. In: (2019). DOI: <https://doi.org/10.1109/SP.2019.00089>. (Visited on 11/26/2025).
 - [52] M. Redeker, B. F. Cockburn and D. G. Elliott. “An investigation into crosstalk noise in DRAM structures”. In: (2002). DOI: [10.1109/MTDT.2002.1029773](https://doi.org/10.1109/MTDT.2002.1029773). (Visited on 02/06/2026).
 - [53] Mark Seaborn, and Thomas Dullien. “Exploiting the DRAM rowhammer bug to gain kernel privileges”. In: (2015). URL: <https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges-wp.pdf%20https://projectzero.google/2015/03/exploiting-dram-rowhammer-bug-to-gain.html> (visited on 12/29/2025).
 - [54] Martin Heckel, and Florian Adamsky. “Flipper: Rowhammer on Steroids”. In: (2025). DOI: <https://doi.org/10.46586/uasc.2025.002>. (Visited on 11/26/2025).
 - [55] Martin Heckel, and Florian Adamsky. “Reverse-Engineering Bank Addressing Functions on AMD CPUs”. In: (2023). URL: <https://dramsec.ethz.ch/papers/reveengamd.pdf> (visited on 01/15/2026).
 - [56] Matthew Blackmore. “A Quantitative Analysis of Memory Controller Page Policies”. Masterthesis. Portland State University, 2013. URL: https://pdxscholar.library.pdx.edu/cgi/viewcontent.cgi?article=1659&context=open_access_etds (visited on 11/26/2025).

-
- [57] mdn. *Fuzz-Tests*. URL: <https://developer.mozilla.org/de/docs/Glossary/Fuzzing> (visited on 02/02/2026).
 - [58] Michael Bechtel, and Heechul Yun. “Memory-Aware Denial-of-Service Attacks on Shared Cache in Multicore Real-Time Systems”. In: (2021). DOI: <https://doi.org/10.1109/TC.2021.3108044>. (Visited on 11/26/2025).
 - [59] Michael Schwarz. “DRAMA: Exploiting DRAM Buffers for Fun and Profit”. Masterthesis. Graz University of Technology, 2016. URL: <https://blackhat.com/docs/eu-16/materials/eu-16-Schwarz-How-Your-DRAM-Becomes-A-Security-Problem-wp.pdf> (visited on 01/28/2026).
 - [60] Micron. *LPDDR4X/LPDDR4 SDRAM*. URL: https://www.mouser.com/datasheet/2/671/z4bm_embedded_lpddr4x_lpddr4-3193428.pdf?srsltid=AfmB0oosiPjdpHJFHgsNPLezD5lZeK0Vsm-CQFsx3nFMOKjtvkmCLpQ6 (visited on 11/26/2025).
 - [61] Minghua Wang, Zhi Zhang, Yueqiang Cheng, and Surya Nepal. “DRAMDig: A Knowledge-assisted Tool to Uncover DRAM Address Mapping”. In: (2020). URL: <https://arxiv.org/pdf/2004.02354> (visited on 11/26/2025).
 - [62] Moritz Lipp. “Cache Attacks and Rowhammer on ARM”. Masterthesis. TU Graz, 2016. URL: https://mlq.me/download/master_thesis.pdf (visited on 11/26/2025).
 - [63] Moritz Lipp, Daniel Gruss, Raphael Spreitzer, Clémentine Maurice, and Stefan Mangard. “ARMageddon: Cache Attacks on Mobile Devices”. In: (2016). URL: https://www.usenix.org/system/files/conference/usenixsecurity16/sec16_paper_lipp.pdf (visited on 11/26/2025).
 - [64] Moritz Lipp, Misiker Tadesse Aga, Michael Schwarz, Daniel Gruss, Clémentine Maurice, Lukas Raab, and Lukas Lamster. “Nethammer: Inducing Rowhammer Faults through Network Requests”. In: (2018). DOI: <https://doi.org/10.48550/arXiv.1805.04956>. (Visited on 11/26/2025).
 - [65] Onur Mutlu, and Jeremie S. Kim. “RowHammer: A Retrospective”. In: (2019). DOI: <https://doi.org/10.48550/arXiv.1904.09724>. (Visited on 11/26/2025).
 - [66] Patrick Jattke, Victor van der Veen, Pietro Frigo, Stijn Gunter, and Kaveh Razavi. “BLACKSMITH: Scalable Rowhammering in the Frequency Domain”. In: (2022). DOI: <https://doi.org/10.1109/SP46214.2022.9833772>. (Visited on 11/26/2025).
 - [67] Peter Pessl, Daniel Gruss, Clémentine Maurice, Michael Schwarz, and Stefan Mangard. “DRAMA: Exploiting DRAM Addressing for Cross-CPU Attacks”. In: (2016). URL: <https://gruss.cc/files/drama.pdf> (visited on 11/26/2025).
 - [68] Pierre Chor-Fung Chia, Shi-Jie Wen, and Sanghyeon Baeg. “New DRAM HCI qualification method emphasizing on repeated memory access”. In: (2010). DOI: [10.1109/IIRW.2010.5706509](https://doi.org/10.1109/IIRW.2010.5706509). (Visited on 02/06/2026).
 - [69] Pietro Frigo, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. “Grand Pwning Unit: Accelerating Microarchitectural Attacks with the GPU”. In: (2018). DOI: <https://doi.org/10.1109/SP.2018.00022>. (Visited on 01/28/2026).
 - [70] Pietro Frigo, Emanuele Vannacci, Hasan Hassan, Victor van der Veen, Onur Mutlu, Cristiano Giuffrida, Herbert Bos, and Kaveh Razavi. “TRRespass: Exploiting the Many Sides of Target Row Refresh”. In: (2020). DOI: <https://doi.org/10.1109/SP40000.2020.00090>. (Visited on 11/26/2025).

-
- [71] PowerCert Animated Videos. *CPU Cache Explained - What is Cache Memory?* 2016. URL: <https://youtu.be/yi0FhRqDJfo?si=S9AKrismPQv5NNNB> (visited on 11/26/2025).
 - [72] Raphael Spreitzer, and Benoit Gérard. “Towards More Practical Time-Driven Cache Attacks”. In: (2014). DOI: 10.1007/978-3-662-43826-8_3. (Visited on 01/12/2026).
 - [73] Raphael Spreitzer, and Thomas Plo. “Cache-Access Pattern Attack on Disaligned AES T-Tables”. In: (2013). DOI: 10.1007/978-3-642-40026-1_13. (Visited on 01/12/2026).
 - [74] Raspberry Pi. *Processors*. URL: <https://www.raspberrypi.com/documentation/computers/processors.html> (visited on 11/26/2025).
 - [75] Rui Qiao, and Mark Seaborn. “A New Approach for Rowhammer Attacks”. In: (2016). DOI: <https://doi.org/10.1109/HST.2016.7495576>. (Visited on 11/26/2025).
 - [76] S. Zeitouni, D. Gens, and A. -R. Sadeghi. “It’s Hammer Time: How to Attack (Rowhammer-based) DRAM-PUFs”. In: (2018). DOI: 10.1109/DAC.2018.8465890. (Visited on 02/12/2026).
 - [77] Stacey Peterson, Garry Kranz, and Jon Toigo. *Die Unterschiede zwischen Cache und RAM erklärt*. 2025. URL: <https://www.computerweekly.com/de/ratgeber/Was-ist-der-Unterschied-zwischen-Cache-und-RAM> (visited on 11/26/2025).
 - [78] Sumax. *Cache*. URL: <https://www.sumax.de/wissen/lexikon/cache.html> (visited on 11/26/2025).
 - [79] sv:io. *LPDDR5 Tutorial: Deep dive into its physical structure*. URL: <https://www.systemverilog.io/design/lpddr5-tutorial-physical-structure/#bank-rows-and-columns> (visited on 02/06/2026).
 - [80] T. Chao, Hao-Yu Yang, Rei-Fu Huang, Shih-Chin Lin, and Ching-Yu Chin. “Fault models for embedded-DRAM macros”. In: (2009). DOI: 10.1145/1629911.1630097. (Visited on 02/06/2026).
 - [81] Victor van der Veen, Martina Lindorfer, and Yanick Fratantonio. “GuardION: Practical Mitigation of DMA-Based Rowhammer Attacks on ARM”. In: (2018). DOI: 10.1007/978-3-319-93411-2_5. (Visited on 01/28/2026).
 - [82] Victor van der Veen, Yanick Fratantonio, Martina Lindorfer, Daniel Gruss, Clémentine Maurice, Giovanni Vigna, Herbert Bos, Kaveh Razavi, and Cristiano Giuffrida. “Drammer: Deterministic Rowhammer Attacks on Mobile Platforms”. In: (2016). DOI: <https://doi.org/10.1145/2976749.2978406>. (Visited on 11/26/2025).
 - [83] Vikky K. *What is SDRAM Memory*. 2023. URL: <https://embeddedhardwaredesign.com/what-is-sdram-memory/> (visited on 11/26/2025).
 - [84] vusec. *TRRespass*. 2021. URL: <https://github.com/vusec/trrespass> (visited on 11/26/2025).
 - [85] Yichen Jiang, Huifeng Zhu, Haoqi Shan, Xiaolong Guo, Xuan Zhang, and Yier Jin. “TRRScope: Understanding Target Row Refresh Mechanism for Modern DDR Protection”. In: (2022). DOI: <https://doi.org/10.1109/HOST49136.2021.9702274>. (Visited on 11/26/2025).

-
- [86] Yohannes Bekele, Ahmed Yiwere, and Daniel Limbrick. “Rowhammer Attacks on the Raspberry Pi 3B+”. In: (). URL: <https://par.nsf.gov/servlets/purl/10220639> (visited on 11/26/2025).
 - [87] Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, and Konrad Lai Onur Mutlu. “Flipping Bits in Memory Without Accessing Them”. In: (2014). DOI: <https://doi.org/10.1145/2678373.2665726>. (Visited on 11/26/2025).
 - [88] Yuan Xiao, Xiaokuan Zhang, Yinqian Zhang, and Radu Teodorescu. “One Bit Flips, One Cloud Flops: Cross-VM Row Hammer Attacks and Privilege Escalation”. In: (2016). URL: https://www.usenix.org/system/files/conference/usenixsecurity16/sec16_paper_xiao.pdf (visited on 11/26/2025).
 - [89] Yuval Yarom, and Katrina Falkner. “Flush+Reload: A High Resolution, Low Noise, L3 Cache Side-Channel Attack”. In: (2014). URL: <https://www.usenix.org/system/files/conference/usenixsecurity14/sec14-paper-yarom.pdf> (visited on 01/07/2026).
 - [90] Yuval Yarom, and Naomi Benger. “Recovering OpenSSL ECDSA Nonces Using the Flush+Reload Cache Side-channel Attack”. In: (2014). URL: <https://eprint.iacr.org/2014/140.pdf> (visited on 01/07/2026).
 - [91] Z. Al-Ars. “DRAM fault analysis and test generation”. In: (2005). URL: <https://repository.tudelft.nl/record/uuid:93137061-e3ef-418e-aa7a-b7539f70af50> (visited on 02/06/2026).
 - [92] Z. Al-Ars, S. Hamdioui, A. van de Goor, G. Gaydadjiev and J. Vollrath. “DRAM-Specific Space of Memory Tests”. In: (2006). DOI: 10.1109/TEST.2006.297701. (Visited on 02/06/2026).
 - [93] Zelalem Birhanu Aweke, Salessawi Ferede Yitbarek, Rui Qiao, Reetuparna Das, Matthew Hicks, Yossi Oren, and Todd Austin. “ANVIL: Software-Based Protection Against Next-Generation Rowhammer Attacks”. In: (2016). DOI: <https://doi.org/10.1145/2980024.2872390>. (Visited on 11/26/2025).
 - [94] Zhenkai Zhang, Zihao Zhan, Daniel Balasubramanian, Xenofon Koutsoukos, and Gabor Karsai. “Triggering Rowhammer Hardware Fault on ARM: A Revisit”. In: (2018). DOI: <https://doi.org/10.1145/3266444.3266454>. (Visited on 11/26/2025).